

**UNIVERZITET U BEOGRADU
FAKULTET ORGANIZACIONIH NAUKA**

ZAVRŠNI RAD

**Razvoj i evaluacija RAG sistema za
pretraživanje dokumenata**

Mentor

**Dr Sandro Radovanović,
Docent**

Student

Ana Dobrijević 2022/0032

Beograd, 2026. godine

APSTRAKT

Cilj ovog rada je razvoj, implementacija i empirijska evaluacija agentskog sistema za generisanje prošireno pretragom (engl. *Retrieval-Augmented Generation*, RAG [1]) namenjenog pretraživanju i odgovaranju na pitanja nad obimnim tekstualnim dokumentom. Sistem je projektovan tako da kombinuje tri različite strategije pronalaženja informacija, leksičku pretragu zasnovanu na algoritmu BM25, gustu vektorsku (semantičku) pretragu zasnovanu na FAISS indeksu i hibridnu pretragu koja te dve strategije objedinjuje postupkom recipročne fuzije rangova (engl. *Reciprocal Rank Fusion*, RRF). Svaka strategija je upakovana kao alat (engl. *tool*) koji autonomno koristi agent zasnovan na ReAct paradigmi (engl. *Reasoning + Acting*), čime se klasična, statična RAG cevovodna arhitektura proširuje u agentsku, iterativnu arhitekturu.

Kao testni korpus korišćena je knjiga *Speech and Language Processing* [19], treće izdanje (javno dostupna radna verzija), obima preko 600 strana. Dokument je podeljen na segmente fiksne veličine (512 tokena uz preklapanje od 64 tokena), vektorizovan modelom *text-embedding-3-small* i indeksiran paralelno u FAISS i BM25 indeks. Za jezičko zaključivanje i generisanje korišćen je model *GPT-4o-mini*. Evaluacija je sprovedena standardizovanim okvirom RAGAS nad sintetički generisanim test skupom od 30 pitanja, pri čemu su mereni vernost (engl. *faithfulness*), relevantnost odgovora (engl. *answer relevancy*), odziv konteksta (engl. *context recall*) i preciznost konteksta (engl. *context precision*).

Rezultati pokazuju da hibridna strategija ostvaruje najviše prosečne vrednosti na sve četiri metrike (*faithfulness* 0,881; *answer relevancy* 0,887; *context recall* 0,900; *context precision* 0,900), nadmašujući i leksičku i vektorsku strategiju posmatrane pojedinačno, a jedina je strategija bez ijednog slučaja potpunog izostanka konteksta. Leksička pretraga (BM25) pokazala se neočekivano snažnom osnovicom, dok je samostalna vektorska pretraga, u datoj konfiguraciji i uz datu infrastrukturu, imala najslabije rezultate. Rad pruža matematičku formalizaciju primenjenih metoda, detaljnu analizu izvornog koda implementacije i interpretaciju empirijskih rezultata, uključujući analizu po pojedinačnim pitanjima i ispitivanje postavljenih hipoteza.

Ključne reči: generisanje prošireno pretragom (RAG), agentski sistemi, ReAct, BM25, vektorska pretraga, FAISS, hibridna pretraga, recipročna fuzija rangova, RAGAS, evaluacija, veliki jezički modeli.

SADRŽAJ

1. UVOD	5
2. TEORIJSKE OSNOVE	8
2.1 Veliki jezički modeli.....	8
2.2 Generisanje prošireno pretragom (RAG).....	12
2.3 Strategije pronalaženja informacija	17
2.4 Agentski sistemi i ReAct paradigma.....	25
2.5 Evaluacija RAG sistema, RAGAS okvir	29
3. PREGLED LITERATURE	34
3.1 Razvoj i generacije RAG paradigme	34
3.2 Strategije pronalaženja i njihova fuzija.....	34
3.3 Agentski sloj i evaluacija	35
3.4 Pozicioniranje rada u odnosu na literaturu.....	35
3.5 Detaljan osvrt na ključne radove.....	36
3.6 Komparativna analiza i pozicioniranje u odnosu na novije pravce	37
4. PROJEKTOVANJE SISTEMA.....	39
4.1 Arhitektura sistema	39
4.2 Modul za pripremu podataka	42
4.3 Modul za indeksiranje (fabrika alata)	42
4.4 Agentski orkestrator.....	43
4.5 RAGAS evaluator	43
4.7 Projektni obrasci i načela	44
4.8 Strategija obrade grešaka i otpornosti.....	45
5. IMPLEMENTACIJA.....	47
5.1 Tehnološki stek	47
5.2 Razvojno okruženje	47
5.3 Implementacija po modulima	47

5.4 Problemi tokom implementacije	52
5.5 Analiza modula za konfiguraciju	53
5.6 Formatiranje konteksta i navođenje izvora	53
5.7 Komandno-linijski tok i ponovljivost	54
5.8 Detaljan prikaz izgradnje vektorskog indeksa	55
5.9 Priprema skupa podataka za evaluaciju	55
6. EKSPERIMENTALNA EVALUACIJA	57
6.1 Eksperimentalna postavka	57
6.2 Rezultati po metrikama	59
6.3 Analiza rezultata	66
6.4 Provera hipoteza.....	67
6.5 Studije slučaja pojedinačnih pitanja.....	68
6.6 Sintetički pregled obrazaca	71
6.9 Diskusija o izboru pondera fuzije α	72
7. DISKUSIJA	74
7.1 Interpretacija rezultata	74
7.2 Ograničenja istraživanja.....	74
7.3 Budući pravci	75
7.4 Ekonomija sistema: trošak i performanse	75
8. ZAKLJUČAK	77
LITERATURA	78

1. UVOD

Veliki jezički modeli (engl. *Large Language Models*, LLM) poslednjih godina postigli su značajne rezultate u oblasti automatske obrade prirodnog jezika. Modeli iz porodica GPT, Claude, Gemini i LLaMA pokazuju visok stepen uspešnosti u razumevanju, sažimanju, prevođenju i generisanju koherentnog teksta na većem broju jezika. Međutim, ovi modeli imaju jedno suštinsko ograničenje. Njihovo znanje sadržano je u parametrima utvrđenim tokom predtreniranja i ograničeno je na podatke dostupne do datuma zaključenja korpusa za obuku (engl. [10] *knowledge cutoff*). Kada se modelu postavi pitanje o sadržaju koji nije obuhvaćen tim parametrima, na primer o internom poslovnom dokumentu, novijem naučnom radu ili usko specijalizovanoj temi, model ne raspolaže mehanizmom da takav sadržaj pribavi iz spoljašnjeg izvora, već odgovor izvodi isključivo na osnovu naučenih obrazaca.

Posledica navedenog ograničenja jeste pojava halucinacija (engl. *hallucinations* [7]), pri kojima model generiše jezički tečan i naizgled ubedljiv odgovor koji sadrži netačne ili nepostojeće podatke. Tipični primeri uključuju izmišljene datume, nepostojeće autore, naslove radova i bibliografske navode. U domenima u kojima se odluke donose na osnovu pribavljenih informacija, kao što su pravo, medicina, finansije i tehnička dokumentacija, halucinacije predstavljaju ozbiljnu prepreku za primenu jezičkih modela. Iz tog razloga njihova samostalna upotreba u takvim oblastima zahteva dodatne mehanizme kojima se obezbeđuje proverljivost odgovora.

Paradigma proširenog generisanja pretragom (RAG) jedan je od sistemskih pristupa rešavanju ovog problema. Umesto da se oslanja isključivo na parametarsko znanje, sistem pri postavljanju pitanja najpre pretražuje spoljašnju bazu znanja i izdvaja segmente koji su semantički ili leksički najsrodniji pitanju. Ti segmenti se zatim prosleđuju modelu kao deo konteksta, na osnovu kojeg model formira odgovor. Time odgovor postaje utemeljen (engl. *grounded*) u konkretnim izvorima, čime se smanjuje verovatnoća halucinacija i omogućava provera tačnosti praćenjem odgovora do izvornog dokumenta.

Kvalitet RAG sistema u velikoj meri zavisi od kvaliteta komponente za pronalaženje informacija, koja se naziva pretraživačem (engl. *retriever* [8]). Ako pretraživač ne izdvoji relevantne segmente, ni jezički model visokih sposobnosti ne može iz neodgovarajućeg konteksta izvesti tačan odgovor. Odatle sledi osnovno istraživačko pitanje ovog rada: koja strategija pronalaženja informacija obezbeđuje najpouzdanije odgovore i da li, i u kojoj meri,

kombinacija više komplementarnih strategija (hibridizacija) donosi merljivu prednost u odnosu na pojedinačne pristupe.

Poseban aspekt razmatranja jeste agentski pristup. Klasičan RAG izvršava pretragu jednom, unapred i po fiksnoj strategiji. Agentski RAG, nasuprot tome, prepušta jezičkom modelu da samostalno odluči koji alat za pretragu će pozvati, sa kojim upitom i koliko puta. Takva fleksibilnost povećava sposobnost sistema da odgovori na složenija pitanja, ali istovremeno otvara pitanja stabilnosti, ponovljivosti i predvidljivosti, koja se u ovom radu posebno razmatraju. [2]

Predmet istraživanja je hibridni RAG sistem upravljani autonomnim agentom, koji nad jednim obimnim dokumentom kombinuje leksičku, vektorsku i hibridnu pretragu i odgovara na pitanja prirodnog jezika.

Cilj istraživanja je empirijska, kvantitativna evaluacija tri navedene strategije pronalaženja informacija pomoću standardizovanog RAGAS okvira, na osnovu koje se izводе preporuke o izboru strategije u praktičnim sistemima. Sekundarni cilj je formalna i implementaciona dokumentacija celokupnog sistema, tako da rezultati budu ponovljivi i proverljivi.

Iz ovako postavljenog cilja izvedena su tri istraživačka pitanja:

1. **IP1:** Da li hibridna pretraga nadmašuje pojedinačne strategije (BM25 i vektorsku) posmatrane zasebno?
2. **IP2:** Kako autonomni ReAct agent, koji samostalno bira alat za pretragu, utiče na kvalitet i stabilnost odgovora?
3. **IP3:** Koje RAGAS metrike najpouzdanije razlikuju kvalitet strategija na datom korpusu i datom test skupu?

Na osnovu pregleda literature i prirode primenjenih algoritama postavljene su tri istraživačke hipoteze:

- **H1:** Hibridni pretraživač postiže više vrednosti metrike faithfulness od BM25 i vektorske pretrage posmatranih pojedinačno.
- **H2:** Vektorska pretraga ima prednost nad BM25 kod semantički složenih pitanja (parafraze, pojmovna pitanja bez doslovnog poklapanja termina).
- **H3:** ReAct agent koji ima na raspolaganju više alata postiže višu context precision od agenta ograničenog na jedan alat.

Hipoteze se proveravaju u poglavlju 6 na osnovu izmerenih vrednosti, uz svest o ograničenjima sintetičkog test skupa o kojima se diskutuje u poglavlju 7.

Da bismo odgovorili na istraživačka pitanja i ispitali postavljene hipoteze, sprovedeno je istraživanje u tri međusobno povezane faze. Istraživanje je sprovedeno u tri međusobno povezane faze. Prva faza je pregled relevantne naučne literature iz oblasti RAG sistema, strategija pronalaženja informacija, agentskih sistema i automatizovane evaluacije. Druga faza je projektovanje i implementacija softverskog sistema u programskom jeziku Python, uz upotrebu biblioteka LangChain, FAISS, rank-bm25, RAGAS i Gradio. Treća faza je empirijska evaluacija sistema na sintetički generisanom test skupu, uz statističku obradu i interpretaciju rezultata.

Metodološki, rad pripada eksperimentalnoj računarskoj nauci: definiše se merljiva hipoteza, konstruiše se kontrolisani eksperiment (iste postavke, isti test skup, ista temperatura modela) i porede se tri tretmana (BM25, vektorska, hibridna pretraga) po istim metrikama.

Rad je organizovan u osam poglavlja. Nakon uvoda, drugo poglavlje izlaže teorijske osnove, velike jezičke modele, RAG paradigmu, strategije pretraživanja, agentske sisteme i RAGAS okvir, uz matematičku formalizaciju ključnih veličina. Treće poglavlje daje pregled literature. Četvrto poglavlje opisuje projektovanje sistema i njegovu arhitekturu. Peto poglavlje detaljno prikazuje implementaciju po modulima, sa analizom izvornog koda. Šesto poglavlje sadrži eksperimentalnu evaluaciju i rezultate. Sedmo poglavlje donosi diskusiju, ograničenja i buduće pravce, a osmo poglavlje izvodi zaključke. Na kraju se nalazi literature.

2. TEORIJSKE OSNOVE

U ovom poglavlju izlažu se teorijski koncepti na kojima počiva implementirani sistem. Cilj je da se uspostavi pojamovni i matematički okvir potreban za razumevanje arhitekture, implementacije i rezultata evaluacije.

2.1 Veliki jezički modeli

2.1.1 Transformer arhitektura

Veliki jezički modeli zasnovani su na transformer arhitekturi [10]. Ključna inovacija transformera je mehanizam samopažnje (engl. *self-attention*), koji svakom tokenu u sekvenci omogućava da pri formiranju svoje reprezentacije „obradi pažnju“ na sve ostale tokene, ponderisano prema njihovoj relevantnosti. Time se prevazilaze ograničenja rekurentnih mreža u modelovanju dugoročnih zavisnosti i omogućava potpuna paralelizacija obrade sekvence.

Mehanizam pažnje formalno se izračunava nad matricama upita (engl. *Query*, Q), ključeva (engl. *Key*, K) i vrednosti (engl. *Value*, V), koje se dobijaju linearnim projekcijama ulaznih reprezentacija. Skalirana skalarna pažnja definiše se izrazom:

$$Attention(Q, K, V) = softmax(QK^T / \sqrt{d_k}) V \quad (1)$$

Matrice Q , K i V dobijaju se linearnim projekcijama ulaznih reprezentacija X pomoću naučenih matrica težina, odnosno $Q = X \cdot W_q$, $K = X \cdot W_k$ i $V = X \cdot W_v$. Skalarni proizvod $Q \cdot K^T$ daje matricu dimenzije $n \times n$ čiji element na poziciji (i, j) meri usaglašenost upita i -tog tokena sa ključem j -tog tokena. Pošto se ta matrica računa za sve parove tokena, vremenska i prostorna složenost mehanizma pažnje iznosi $O(n^2 \cdot d_k)$, gde je n dužina sekvence. Skaliranje faktorom $1/\sqrt{d_k}$ uvodi se da bi se varijansa skalarnih proizvoda održala na vrednosti reda jedinice; bez njega bi, za velike vrednosti d_k , argumenti funkcije $softmax$ dostizali velike apsolutne vrednosti, čime bi funkcija ušla u zasićenje i proizvela gradijente bliske nuli, što otežava treniranje. Dimenzija ključeva je d_k , a deljenje sa $\sqrt{d_k}$ skalira skalarne proizvode kako bi se sprečile prevelike vrednosti pre primene $softmax$ funkcije. Kod jezičkih modela tipa dekodera, na matricu QK^T primenjuje se kauzalna maska koja gornji trougao matrice postavlja na $-\infty$, čime se postiže da svaki token „vidi“ samo prethodne tokene.

Razlikuju se tri osnovne arhitektonske porodice: enkoderski modeli (npr. BERT-a, dvosmerni, pogodni za razumevanje), dekoderski modeli (autoregresivni, npr. modeli GPT porodice,

pogodni za generisanje) i enkoder-dekoder modeli (pogodni za zadatke preslikavanja sekvence u sekvencu, poput prevođenja). Model korišćen u ovom radu, GPT-4o-mini, pripada dekoderskoj porodici.

Transformer arhitektura je razvijena kao odgovor na ograničenja rekurentnih neuronskih mreža (RNN), LSTM i GRU modela. One, uključujući naprednije varijante poput LSTM i GRU, obrađuju sekvencu token po token, prosleđujući skriveno stanje sa jednog koraka na sledeći. Ovaj sekvencijalni postupak ima dva fundamentalna nedostatka. Prvo, on je inherentno serijski, obrada n-tog tokena zahteva da prethodno budu obrađeni svi tokeni pre njega, što onemogućava paralelizaciju na savremenom grafičkom hardveru. Drugo, informacija sa početka duge sekvence mora da „preživi“ prolaz kroz mnogo koraka da bi uticala na kasnije reprezentacije, a signal pritom slabi (problem nestajućeg gradijenta). [10]

Transformer rešava oba problema istovremeno. Pošto mehanizam pažnje svakom tokenu daje direktan pristup svim ostalim tokenima, dužina puta informacije između bilo koja dva tokena je konstantna ($O(1)$ koraka), nezavisno od njihove udaljenosti u sekvenci. Istovremeno, pošto se pažnja nad svim parovima tokena može izračunati matričnim množenjem, cela operacija je potpuno paralelizovana. Cena je kvadratna složenost po dužini sekvence, $O(n^2)$, jer se računa interakcija svakog para tokena, što je i motivacija za kasnija istraživanja efikasnijih varijanti pažnje.

Mehanizam pažnje retko se primenjuje kao jedna funkcija; umesto toga koristi se višeglava pažnja (engl. *multi-head attention*), u kojoj se projekcije Q, K i V dele na h nezavisnih „glava“, od kojih svaka uči da obraća pažnju na drugačiji aspekt odnosa među tokenima (npr. sintaksičke zavisnosti, koreferencu, lokalni kontekst). Izlazi glava se spajaju i ponovo projektuju:

$$MultiHead(Q, K, V) = Concat(head_1, \dots, head_h) W^O \quad (2)$$

gde je svaka glava $head_i = Attention(Q W_i(Q), K W_i(K), V W_i(V))$. Pored sloja pažnje, svaki transformer blok sadrži poziciono potpuno povezanu mrežu (engl. *feed-forward network*), rezidualne veze i normalizaciju sloja (engl. *layer normalization*), koje zajedno stabilizuju i ubrzavaju treniranje dubokih mreža. Pošto sam mehanizam pažnje ne razlikuje redosled tokena, dodaju se pozicione ugradnje (engl. *positional encodings*) koje modelu prenose informaciju o poretku.

U okviru eksperimentalne evaluacije analizirano je i pitanje koje se odnosi na kauzalno maskiranje u matrici QK^T dimenzije $N \times N$. Tačan odgovor podrazumeva identifikaciju

elemenata iznad glavne dijagonale koji se postavljaju na vrednost $-\infty$ kako bi se onemogućio pristup budućim tokenima. Ovaj primer pokazuje značaj preciznog pronalaženja relevantnih informacija iz tehničkog sadržaja korpusa.

Paradigma rada sa ovim modelima obuhvata predtreniranje (engl. *pre-training*) na ogromnim tekstualnim korpusima uz funkciju gubitka unakrsne entropije, nakon čega slede fino podešavanje na instrukcijama (engl. *instruction tuning*) i poravnanje sa ljudskim preferencijama (engl. *preference alignment*).

GPT-4o-mini je optimizovana, ekonomična varijanta GPT-4o porodice. Posедуje veliki kontekstni prozor (do 128K tokena), što ga čini pogodnim za RAG scenarije u kojima se modelu prosleđuje više pronađenih segmenata zajedno sa pitanjem. Model dobro razume i generiše tekst, podržava pozivanje funkcija (engl. *function calling*) i deterministički rad pri temperaturi 0, što je iskorišćeno u ovom radu radi ponovljivosti. Njegova ograničenja su uobičajena za jezičke modele: zamrznuto znanje i sklonost halucinacijama u odsustvu konteksta.

2.1.3 Matematika softmax funkcije i temperatura

Jezgro i mehanizma pažnje i generisanja tokena jeste softmax funkcija, koja vektor realnih brojeva preslikava u verovatnosnu raspodelu. Za vektor logita $z = (z_1, \dots, z_n)$, softmax je definisan kao:

$$\text{softmax}(z_i) = e^{(z_i)} / \sum_j e^{(z_j)} \quad (3)$$

Funkcija ima dva ključna svojstva. Prvo, svi izlazi su pozitivni i sumiraju se na 1, pa čine ispravnu verovatnosnu raspodelu. Drugo, ona je „meka“ verzija funkcije maksimuma: najveći logit dobija najveću verovatnoću, ali i ostali zadržavaju nenultu masu, što omogućava diferencijabilnost neophodnu za treniranje gradijentnim spustom. U mehanizmu pažnje, softmax se primenjuje na skalirane skalarne proizvode $QK^T/\sqrt{d_k}$, čime se dobijaju težine pažnje koje sumiraju na 1 i koje određuju koliko svaki token doprinosi izlaznoj reprezentaciji.

Pri generisanju tokena uvodi se temperatura T koja deli logite pre softmax-a:

$$p(z_i; T) = e^{(z_i / T)} / \sum_j e^{(z_j / T)} \quad (4)$$

Pri $T \rightarrow 0$ raspodela se zaoštava ka determinističkom izboru najverovatnijeg tokena dok pri $T > 1$ raspodela postaje ravnija i uzorkovanje raznovrsnije. Skaliranje skalarnih proizvoda faktorom $1/\sqrt{d_k}$ u mehanizmu pažnje ima srodnu ulogu: bez njega bi, za velike d_k , skalarni

proizvodi imali veliku varijansu, gurajući softmax u režim zasićenja sa izuzetno malim gradijentima i otežanim treniranjem.

2.1.4 Problem halucinacija

Halucinacija je generisani sadržaj koji nije utemeljen u ulaznim podacima niti u proverljivim činjenicama. Razlikuju se halucinacije vernosti, kod kojih odgovor protivreči datom kontekstu i halucinacije činjeničnosti, kod kojih odgovor protivreči stvarnom stanju. U kontekstu RAG sistema relevantnija je vernost, to jest pitanje da li je svaka tvrdnja u odgovoru poduprta pronađenim kontekstom; tu osobinu meri metrika vernosti, opisana u odeljku 2.5. RAG predstavlja sistemsko rešenje problema halucinacija jer odgovor uslovljava spoljašnjim, proverljivim izvorima.

2.1.5 Životni ciklus i režimi rada jezičkog modela

Savremeni jezički model prolazi kroz nekoliko faza pre nego što postane upotrebljiv u sistemu poput ovog. U fazi predtreniranja, model uči da predviđa sledeći token nad ogromnim korpusom teksta, minimizujući unakrsnu entropiju. Ova faza modelu daje široko, ali nediferencirano znanje o jeziku i svetu. U fazi finog podešavanja na instrukcijama, model se izlaže parovima instrukcija i željenih odgovora, čime uči da prati zahteve korisnika. U fazi poravnanja, najčešće učenjem pojačanjem iz ljudskih povratnih informacija (engl. *RLHF*), model se dodatno usmerava ka korisnim, bezbednim i istinitim odgovorima.

U RAG sistemu, model se koristi isključivo u režimu zaključivanja (engl. *inference*), bez ikakvog daljeg treniranja. Posledica je da se sve „znanje“ koje se odnosi na konkretni dokument ne nalazi u parametrima modela, već se isporučuje kroz kontekst u trenutku upita. Time se odgovornost za tačnost prebacuje sa parametarskog znanja (koje je zamrznuto i neproverljivo) na pronađeni kontekst (koji je svež i proverljiv), što je značajna prednost RAG paradigme i razlog zbog kojeg je kvalitet pretraživača presudan.

Temperatura uzorkovanja je hiperparametar koji kontroliše slučajnost generisanja. Pri visokim vrednostima (npr. 0,8) model bira tokene raznovrsnije i kreativnije; pri temperaturi 0 model je deterministički i uvek bira najverovatniji token. U ovom radu temperatura se postavlja na 0 svuda gde je potrebna ponovljivost (odgovaranje, evaluacija), a na umerenu vrednost 0,3 samo pri generisanju test pitanja, gde je poželjna izvesna raznovrsnost.

2.1.6 Funkcija gubitka i jezičko modelovanje

Predtreniranje jezičkog modela svodi se na zadatak predviđanja sledećeg tokena. Za korpus tokena ($w_1, \dots, w_T$), model uči da maksimizuje verovatnoću svakog tokena uslovljenu prethodnim tokenima, što je ekvivalentno minimizaciji negativne log-verodostojnosti, odnosno unakrsne entropije:

$$L = - (1/T) \sum_t \log p(w_t | w_1, \dots, w_{t-1}) \quad (5)$$

Ova funkcija gubitka ima jasnu informaciono-teorijsku interpretaciju: ona meri prosečan broj bitova potrebnih da se kodira svaki token pod modelom raspodele. Njena eksponencirana vrednost je perpleksnost (engl. *perplexity*), standardna mera kvaliteta jezičkog modela, niža perpleksnost znači da model manje „iznenađuje“ stvarni tekst. Jedno od test pitanja u eksperimentu direktno se odnosi na ovaj pojam (pitanje o signalu treniranja u jezičkom modelovanju), gde je tačan odgovor upravo sledeći token kao implicitna oznaka, što povezuje teorijsku postavku sa konkretnim sadržajem korpusa.

2.2 Generisanje prošireno pretragom (RAG)

2.2.1 Osnovna RAG arhitektura

Osnovni RAG cevovod, formalizovan u radu [1], sastoji se iz dva koraka. U prvom koraku, na osnovu korisničkog pitanja, komponenta za pronalaženje (retriever) iz baze znanja izdvaja najrelevantnijih K segmenata. U drugom koraku, jezički model (generator) generiše odgovor uslovljen pitanjem i pronađenim segmentima. Shematski, tok je sledeći: korisnik postavlja pitanje, pretraživač vraća top- K dokumenata, a generator na osnovu njih formira odgovor.

Formalno, ako sa q označimo pitanje, sa D bazu segmenata i sa $R(q, D)$ skup pronađenih segmenata, odgovor a se modeluje uslovnom verovatnoćom:

$$p(a | q) = \sum p(a | q, d) \cdot p(d | q) \quad (6)$$

gde se sumiranje vrši po pronađenim segmentima $d \in R(q, D)$. U praktičnim implementacijama, kao u ovom radu, pronađeni segmenti se jednostavno konkatenuiraju u kontekst i prosleđuju modelu, čime se gornji izraz aproksimira.

U praktičnim implementacijama pronađeni segmenti se često spajaju u jedinstven kontekst koji se zajedno sa upitom prosleđuje modelu. Kada se K segmenata $d_1, \dots, d_K$ spoji u jedinstven

kontekst i prosledi modelu zajedno sa pitanjem, model nad tim proširenim ulazom obavlja pažnju i implicitno odmerava doprinos svakog segmenta. Time se marginalizacija po dokumentima iz formalnog izraza aproksimira jednim prolazom kroz model, što je računski daleko jeftinije od zasebnog generisanja za svaki segment i potonjeg kombinovanja. Cena je da model mora da raspolaze dovoljno velikim kontekstnim prozorom da primi sve segmente, uslov koji GPT-4o-mini, sa prozorom do 128K tokena, lako ispunjava za $K = 5$ segmenata veličine 512 tokena.

Kvalitet sistema zavisi od dve komponente: kvaliteta pronalaženja informacija, odnosno toga da li su odgovarajući segmenti uključeni u kontekst, i kvaliteta generisanja odgovora, odnosno toga da li model na osnovu tačnog konteksta formira tačan odgovor. Ako se kvalitet pronalaženja označi sa P_r , a uslovni kvalitet generisanja sa P_g , ukupni kvalitet približno je jednak proizvodu $P_r \cdot P_g$. Nijedna komponenta ne nadoknađuje potpun izostanak druge: savršen generator nad praznim kontekstom daje halucinaciju, dok pretraživač visoke tačnosti uz slab generator daje netačan odgovor i pored dobrog konteksta. U sprovedenom eksperimentu faktor P_r pokazao se kao glavni izvor razlika između strategija, jer su strategije koje su češće izostavljale kontekst ($P_r \rightarrow 0$) imale znatno niže ukupne rezultate.

2.2.2 Naivni, napredni i modularni RAG

U literaturi [7] razlikuju se tri generacije RAG sistema. Naivni RAG je jednostavan, linearan cevovod (podela na segmente $\rightarrow$ indeksiranje $\rightarrow$ pretraga $\rightarrow$ generisanje). Napredni RAG uvodi optimizacije pre i posle pretrage (preformulacija upita, ponovno rangiranje, kompresija konteksta). Modularni RAG tretira komponente kao zamenljive module koji se mogu kombinovati i preuređivati. Sistem razvijen u ovom radu pripada modularnom pristupu: strategije pretrage su zamenljivi alati, a agentski sloj odlučuje koji modul se aktivira.

2.2.3 Istorijski razvoj i motivacija RAG-a

RAG paradigma nastala je kao odgovor na ograničenja ranijih pristupa integraciji eksternog znanja u jezičke modele. Pre pojave RAG-a, postojala su dva nezadovoljavajuća pristupa obogaćivanju jezičkih modela znanjem. Prvi pristup bilo je fino podešavanje modela na domenskim podacima, ali ono je skupo, zahteva ponovno treniranje pri svakoj promeni baze znanja, i ne rešava problem proverljivosti, jer znanje ostaje rastopljeno u parametrima. Drugi pristup bilo je jednostavno ubacivanje celog dokumenta u kontekst (engl. *stuffing*), ali ono je ograničeno veličinom kontekstnog prozora i postaje neizvodljivo za obimne korpuse.

Na taj način objedinjene su prednosti oba pristupa. Umesto da modelu prosledi celokupno znanje (kao kod ubacivanja celog dokumenta) ili da ga ugradi u parametre (kao kod finog podešavanja), RAG u trenutku upita bira samo relevantne delove baze znanja i prosleđuje ih kao kontekst. Tako se postiže skalabilnost, jer baza znanja može biti proizvoljno velika, ažurnost, jer se baza menja bez ponovnog treniranja modela, i proverljivost, jer se odgovor može pratiti do izvora. Iz tih razloga RAG je postao česta arhitektura za sisteme za odgovaranje na pitanja nad privatnim ili specijalizovanim korpusima.

Komponenta za pronalaženje ima ključnu ulogu u radu sistema. Ona određuje koji podskup baze znanja dospeva pred model. Ako pretraživač izdvoji pogrešne segmente, jezički model ne može da formira tačan odgovor jer nema pristup odgovarajućoj informaciji. Zbog toga je ovaj rad posvećen poređenju strategija pronalaženja, a ne, na primer, poređenju jezičkih modela.

2.2.4 Strategije podele na segmente

Pošto jezički modeli imaju ograničen kontekstni prozor, a indeksiranje se vrši nad manjim jedinicama, dokument se pre indeksiranja deli na segmente (engl. *chunks*). U ovom radu primenjena je podela fiksne veličine: 512 tokena po segmentu uz preklapanje od 64 tokena. Preklapanje sprečava da rečenica koja se nalazi na granici dva segmenta izgubi kontekst. Alternativni pristupi obuhvataju semantičku podelu (granice se određuju prema značenjskim celinama) i hijerarhijsku podelu (segmenti različitih nivoa granularnosti).

```
splitter = RecursiveCharacterTextSplitter(  
    chunk_size=CHUNK_SIZE,  
    chunk_overlap=CHUNK_OVERLAP,  
    separators=["\n\n", "\n", ". ", " ", ""],  
)  
docs = splitter.split_documents(pages)
```

Slika 1: Podela dokumenta na segmente (iz data_preparer.py).

Rekurzivni delilac pokušava da podeli tekst po hijerarhiji separatora, prvo po praznim redovima, zatim po novom redu, rečenicama, razmacima i, u krajnjem slučaju, po karakterima, čime se očuvava prirodna struktura teksta koliko god je to moguće u okviru zadate veličine segmenta.

2.2.5 Modeli za ugradnju (embedding)

Vektorska pretraga zahteva da se tekst preslika u guste vektore (engl. *embeddings*) u kojima semantička sličnost odgovara geometrijskoj blizini. Razvoj ovih modela išao je od statičkih

ugradnji (Word2Vec, GloVe), preko kontekstualnih ugradnji zasnovanih na transformerima (BERT), do savremenih modela posebno treniranih za pronalaženje informacija. U ovom radu korišćen je model text-embedding-3-small kompanije OpenAI, koji proizvodi vektore dimenzije 1536. Svaki segment dokumenta i svaki upit preslikavaju se u isti vektorski prostor, što dozvoljava merenje njihove sličnosti.

2.2.6 Geometrija semantičkog prostora

Vektorska pretraga počiva na pretpostavci distribucione semantike: reči i tekstovi koji se javljaju u sličnim kontekstima imaju slično značenje, pa treba da budu blizu u vektorskom prostoru. Model za ugradnju preslikava svaki tekst u tačku u prostoru visoke dimenzije (ovde 1536 dimenzija). U tom prostoru, semantička srodnost odgovara geometrijskoj blizini, koja se najčešće meri kosinusnom sličnošću, kosinusom ugla između dva vektora.

Kosinusna sličnost se primenjuje umesto euklidskog rastojanja iz preciznog razloga. Ona je nezavisna od dužine (norme) vektora i meri isključivo njihov pravac, što je poželjno jer intenzitet ugradnje često odražava nebitne činioce, poput dužine teksta, dok pravac nosi semantičku informaciju. Za normalizovane vektore jedinične norme kosinusna sličnost i euklidsko rastojanje monotono su povezani, pa pretraga po jednom kriterijumu daje isti poredak kao i po drugom.

Ograničenje ove geometrije, koje se jasno vidi u rezultatima ovog rada, jeste da bliskost u semantičkom prostoru ne garantuje poklapanje na nivou egzaktnih entiteta. Dva segmenta koja govore o sličnoj temi biće bliska, ali ako pitanje traži konkretan broj ili ime koje se pojavljuje u tačno jednom, vektorska pretraga može da ga previdi u korist tematski srodnijih ali činjenično pogrešnih segmenata. Takvo ponašanje može se objasniti osobinama vektorskog prostora, koji semantičku srodnost rangira iznad doslovnog poklapanja entiteta, što je razlog zbog kojeg je vektorska pretraga promašivala faktografska pitanja na kojima je BM25 davao tačne rezultate.

$$\text{sim}(q, d) = E(q) \cdot E(d) / (\|E(q)\| \cdot \|E(d)\|) \in [-1, 1] \quad (7)$$

Vrednost kosinusne sličnosti može se izvesti iz definicije skalarnog proizvoda dva vektora, prema kojoj je $u \cdot v = \|u\| \cdot \|v\| \cdot \cos \theta$, odakle sledi $\cos \theta = (u \cdot v) / (\|u\| \cdot \|v\|)$. Za jedinično normirane vektore, kod kojih je $\|u\| = \|v\| = 1$, kosinusna sličnost svodi se na sam skalarni proizvod $u \cdot v$, a kvadrat euklidskog rastojanja postaje $\|u - v\|^2 = 2 - 2 \cdot (u \cdot v)$. Iz ove veze sledi da je, za normirane vektore, rangiranje po opadajućoj kosinusnoj sličnosti istovetno rangiranju po rastućem

euklidskom rastojanju, pa pretraga najbližih suseda daje isti poredak rezultata bez obzira na to koja se od dve mere koristi. $E(\cdot)$ označava funkciju ugradnje. Vrednost bliska 1 znači visoku semantičku srodnost, vrednost oko 0 nepovezanost, a negativne vrednosti suprotnost. U praksi pretrage zadržavaju se segmenti sa najvišom sličnošću, bez obzira na apsolutni prag.

2.2.7 Kompromisi pri podeli na segmente

Uticaj veličine segmenta razlikuje se između leksičke i vektorske pretrage. Time se uspostavlja odnos između preciznosti i obima konteksta. Mali segmenti (npr. 128 tokena) daju precizne, fokusirane jedinice koje pretraživaču olakšavaju da locira tačno relevantan deo, ali rizikuju da rasparčaju misao koja se proteže kroz više rečenica, ostavljajući model bez dovoljnog konteksta da je razume. Veliki segmenti (npr. 1024 tokena) čuvaju kontekst, ali unose šum, jedan segment može sadržati više tema, pa pretraživač vraća i nerelevantne delove zajedno sa relevantnim, snižavajući preciznost konteksta.

Izbor od 512 tokena u ovom radu daje uravnotežen odnos koji je u praksi pokazao dobre rezultate za tehničke dokumente: dovoljno velik da obuhvati zaokruženu misao ili definiciju, a dovoljno mali da ostane tematski koherentan. Preklapanje od 64 tokena (12,5% veličine segmenta) dodatno ublažava problem graničnih rečenica, ako se ključna informacija nalazi tačno na granici dva segmenta, preklapanje obezbeđuje da se ona u celosti nađe u bar jednom od njih.

Veličina segmenta različito pogađa dve strategije pretrage. BM25, koji se oslanja na učestalost termina, relativno je otporan na veličinu segmenta jer retki termini ostaju prepoznatljivi bez obzira na okolni tekst. Vektorska pretraga je osetljivija: ugradnja velikog, tematski mešovito segmenta je „razmazana“ u semantičkom prostoru i slabije se poklapa sa fokusiranim upitom. Navedena razlika ukazuje na komplementarnost dve strategije, a posredno i na korist njihove fuzije.

Tabela 1: Kompromis veličine segmenta u RAG sistemima.

Veličina segmenta	Preciznost	Kontekst	Rizik
Mala (≈ 128)	Visoka	Nizak	Rasparčana misao
Srednja (≈ 512)	Uravnotežena	Dovoljan	Minimalan (izbor ovog rada)
Velika (≈ 1024)	Niža	Visok	Tematski šum

2.3 Strategije pronalaženja informacija

2.3.1 TF-IDF i algoritam BM25

BM25 (Best Matching 25) je verovatnosna funkcija rangiranja koja procenjuje relevantnost dokumenta D za upit Q na osnovu učestalosti termina, uz korekciju za dužinu dokumenta i retkost termina. Predstavlja zlatni standard leksičke (na ključnim rečima zasnovane) pretrage i, uprkos svojoj jednostavnosti, ostaje izuzetno snažna osnovica [4]. Rezultat za par dokument, upit dat je izrazom:

$$BM25(D, Q) = \sum IDF(q_i) \cdot f(q_i, D) \cdot (k_1 + 1) / (f(q_i, D) + k_1 \cdot (1 - b + b \cdot |D|/avgdl)) \quad (8)$$

Ponašanje funkcije BM25 najlakše se razume kroz granične vrednosti njenih parametara. Kada parametar k_1 teži nuli, doprinos učestalosti termina se gubi, pa se rangiranje svodi na prosto prisustvo ili odsustvo termina, slično binarnom modelu. Kada k_1 teži beskonačnosti, doprinos raste gotovo linearno sa učestalošću, bez zasićenja. Parametar b kontroliše normalizaciju po dužini: pri $b = 0$ dužina dokumenta se zanemaruje, dok pri $b = 1$ normalizacija je potpuna i dokument se kažnjava srazmerno odnosu sopstvene i prosečne dužine. U ovom radu korišćene su podrazumevane vrednosti $k_1 = 1,2$ i $b = 0,75$, koje su se u literaturi pokazale kao robusne na širokom skupu kolekcija. Učestalost termina je $f(q_i, D)$ q_i u dokumentu D , $|D|$ dužina dokumenta, $avgdl$ prosečna dužina dokumenta u kolekciji, a $IDF(q_i)$ inverzna učestalost dokumenta koja kažnjava česte, neinformativne termine. Parametri k_1 i b kontrolišu zasićenje učestalosti termina, odnosno jačinu normalizacije po dužini; u biblioteci `rank-bm25` podrazumevane vrednosti su $k_1 = 1,5$ i $b = 0,75$.

Prednost BM25 je egzaktno poklapanje termina, brzo indeksiranje i nezavisnost od skupih izračunavanja ugradnji. Glavni nedostatak je nerazumevanje semantike, sinonimi i parafraze koje ne dele doslovne termine ostaju nevidljivi za BM25.

```
from langchain_community.retrievers import BM25Retriever
def create_bm25_only_tools(bm25_retriever: BM25Retriever) -> List[Tool]:
    """BM25-only agent - uses ONLY BM25 lexical search."""
    return [
        Tool(name="bm25_search",
            func=lambda q: _fmt(bm25_retriever.get_relevant_documents(q)),
            description="BM25 lexical keyword search. This is the only available tool."),
    ]
```

Slika 2: Izgradnja BM25 pretraživača (iz `tool_factory.py`).

2.3.1.1 Inverzna učestalost dokumenta i intuicija BM25

Efikasnost BM25 metode može se objasniti analizom njenih osnovnih komponenti. Komponenta IDF (engl. *Inverse Document Frequency*) kvantifikuje koliko je neki termin informativan. Termin koji se pojavljuje u gotovo svakom dokumentu (npr. „the“, „is“) ima nisku IDF vrednost i jedva doprinosi rangiranju, dok redak, specifičan termin (npr. „HuBERT“, „HKUST“) ima visoku IDF vrednost i snažno usmerava pretragu. Formalno, u BM25 se najčešće koristi sledeći oblik:

$$IDF(q_i) = \ln((N - n(q_i) + 0.5) / (n(q_i) + 0.5) + 1) \quad (9)$$

gde je N ukupan broj dokumenata (segmenata) u kolekciji, a $n(q_i)$ broj dokumenata koji sadrže termin q_i . Drugi sastavni deo formule jeste zasićenje učestalosti termina: doprinos ponavljanja istog termina u dokumentu raste sve sporije sa porastom učestalosti, čime se sprečava da dokument koji termin ponavlja veliki broj puta dobije nesrazmerno visok skor. Parametar k_1 kontroliše brzinu tog zasićenja, dok parametar b određuje meru u kojoj se dokument kažnjava zbog svoje dužine u odnosu na prosečnu dužinu $avgdl$. Normalizacija po dužini dokumenta i retkosti termina doprinosi dobrim rezultatima BM25 metode, naročito na tehničkim korpusima bogatim specifičnim entitetima.

Takvo ponašanje može se uočiti i na rezultatima dobijenim tokom eksperimenta. Za pitanje o broju transkribovanih razgovora u HKUST Mandarin korpusu, termini „HKUST“, „Mandarin“ i „transcribed“ imaju veoma visoke IDF vrednosti jer se retko javljaju u celom udžbeniku. BM25 stoga gotovo nepogrešivo locira tačno onaj segment koji ih sadrži, postižući savršen skor relevantnosti konteksta. Vektorska pretraga, nasuprot tome, nema garanciju da će gust vektor pitanja biti najbliži upravo tom segmentu, jer semantička blizina ne mora da prati doslovno poklapanje retkih entiteta.

2.3.1.2 Verovatnosno utemeljenje i varijante BM25

Na osnovu opisane funkcije rangiranja, u nastavku se izlaže teorijska osnova koja pokazuje da BM25 nije puka heuristika, već posledica verovatnosnog modela pronalaženja. Funkcija je izvedena iz modela relevantnosti zasnovanog na razdvajanju (engl. *Robertson-Sparck Jones model, RSJ*), formalizovanog u radu [4]. Polazna pretpostavka tog modela jeste da se dokumenti mogu rangirati prema odnosu verovatnoća da dati dokument bude relevantan, odnosno nerelevantan za upit, pri čemu se primenom Bajesove teoreme i pretpostavke o uslovnoj nezavisnosti termina dolazi do zbira težinskih doprinosa pojedinačnih termina upita. Inverzna

učestalost dokumenta je ocena tog odnosa verovatnoća u odsustvu podataka o relevantnosti, čime se objašnjava njen specifičan logaritamski oblik sa pomerajem od 0,5, koji potiče od Laplasove korekcije za retke događaje.

Zasićenje učestalosti termina (engl. *term frequency saturation*) opisuje pojavu da svako naredno ponavljanje istog termina u dokumentu donosi sve manji prirast relevantnosti. Dokument koji termin sadrži deset puta nije deset puta relevantniji od dokumenta koji ga sadrži jednom; nakon određene tačke dodatna ponavljanja gotovo da ne menjaju ocenu. Parametar k_1 kontroliše brzinu tog zasićenja, a hiperbolički oblik funkcije obezbeđuje da doprinos teži graničnoj vrednosti ($k_1 + 1$). Normalizacija po dužini dokumenta (engl. *document length normalization*) ispravlja sistematsku pristrasnost prema dužim dokumentima, koji prosto zbog svoje dužine sadrže više pojavljivanja bilo kog termina. Bez te korekcije, duži segmenti bili bi neopravdano favorizovani; parametar b određuje jačinu korekcije, pri čemu se dužina dokumenta poredi sa prosečnom dužinom u kolekciji ($avgdl$).

U pogledu konkretnih vrednosti parametara, biblioteka rank-bm25, primenjuje podrazumevane vrednosti $k_1 = 1,5$ i $b = 0,75$. Te vrednosti nisu proizvoljne, već predstavljaju empirijski optimum utvrđen na velikom broju standardnih kolekcija u okviru konferencije TREC [4]. Vrednost $b = 0,75$ znači da se normalizacija po dužini primenjuje delimično, a ne potpuno, čime se izbegava prekomerno kažnjavanje srednje dugih dokumenata; vrednost $k_1 = 1,5$ postavlja zasićenje učestalosti na nivo koji se pokazao robusnim na tekstovima različite prirode.

Pored osnovne formulacije, u literaturi postoji nekoliko varijanti razvijenih da isprave njena specifična ograničenja. Varijanta BM25+ uvodi donji prag doprinosa termina kako bi se sprečilo da veoma dugi dokumenti, usled agresivne normalizacije, dobiju doprinos blizak nuli za termine koje ipak sadrže. Varijanta BM25L preraspodeljuje težine u korist dokumenata veće dužine, polazeći od zapažanja da osnovna funkcija prekomerno kažnjava duge ali relevantne dokumente. U ovom radu primenjena je standardna BM25 formulacija, budući da su segmenti ujednačene dužine (512 tokena uz preklapanje), pa problemi koje pomenute varijante rešavaju ovde nemaju značajnog uticaja.

2.3.2 Vektorska pretraga i FAISS

Vektorska (gusta) pretraga upit preslikava u vektor i traži segmente čiji su vektori najbliži vektoru upita. Najčešća mera blizine je kosinusna sličnost, koja meri kosinus ugla između dva vektora:

$$\cos(u, v) = (u \cdot v) / (\|u\| \cdot \|v\|) \quad (10)$$

Za pretragu nad velikim brojem vektora koristi se biblioteka FAISS (Facebook AI Similarity Search) [5]. FAISS nudi egzaktnu pretragu (IndexFlatL2, zasnovan na euklidskom rastojanju) i aproksimativnu pretragu najbližih suseda (npr. HNSW), koja žrtvuje malu tačnost zarad velike brzine. Zbog umerene veličine korpusa i in-memory rada, korišćen je egzaktan FAISS indeks. Glavna prednost vektorske pretrage je razumevanje semantike i otpornost na parafraze; nedostaci su veći memorijski i računski troškovi i zavisnost od kvaliteta modela za ugradnju.

2.3.2.1 Embedding prostori, dimenzionalnost i izbor indeksa

Veza između skalarnog proizvoda i kosinusne sličnosti direktno proizlazi iz definicije skalarnog proizvoda dva vektora, prema kojoj je njihov skalarni proizvod jednak proizvodu njihovih normi i kosinusa ugla između njih:

$$\begin{aligned} u \cdot v &= \|u\| \cdot \|v\| \cdot \cos \theta \Rightarrow \cos \theta = (u \cdot v) / (\|u\| \cdot \|v\|), \quad \|u\| \\ &= \|v\| = 1 \Rightarrow \cos \theta = u \cdot v \end{aligned} \quad (11)$$

Kada su vektori ugradnji prethodno normirani na jediničnu normu, kosinusna sličnost svodi se na sam skalarni proizvod, pa se pretraga može svesti na množenje matrica, što je računski veoma efikasno. Model text-embedding-3-small vraća vektore dimenzije 1536 koji se mogu normirati, čime pretraga postaje ekvivalentna pronalaženju vektora sa najvećim skalarnim proizvodom.

Pitanje zašto vektor od 1536 dimenzija uopšte može da kodira značenje teksta zasniva se na distribucionoj hipotezi (engl. *distributional hypothesis*), prema kojoj se značenje reči određuje kontekstima u kojima se ona pojavljuje, sažeto u poznatoj formulaciji da se reč prepoznaje po društvu koje drži [13]. Tokom predtreniranja, model uči da rečima i odseccima teksta koji se javljaju u sličnim kontekstima dodeli bliske vektore. Pojedinačne dimenzije vektora ne odgovaraju unapred zadatim, svojstvima koje je moguće da tumače ljudi, već latentnim obeležjima koja zajednički kodiraju semantičke i sintaksne odnose; semantička sličnost dva teksta tako postaje geometrijska blizina njihovih vektora.

Rad sa vektorima visoke dimenzionalnosti otvara pojavu poznatu kao prokletstvo dimenzionalnosti (engl. *curse of dimensionality*). Sa porastom broja dimenzija, euklidska rastojanja između proizvoljnih tačaka teže da se izjednače, pa razlika između najbližeg i najdaljeg suseda postaje sve manja, čime euklidska distanca gubi diskriminativnu moć. Kosinusna sličnost ostaje pouzdanija jer meri pravac, a ne apsolutni položaj vektora, pa nije

osetljiva na koncentraciju normi koja prati visoku dimenzionalnost. Iz tog razloga se u pronalaženju zasnovanom na ugradnjama gotovo isključivo koristi kosinusna sličnost.

Biblioteka FAISS nudi više tipova indeksa, među kojima su za ovaj rad bili relevantni egzaktni indeks zasnovan na potpunom pretraživanju (IndexFlatL2) i aproksimativni indeks zasnovan na navigaciji po hijerarhijskom grafu malih svetova (engl. *Hierarchical Navigable Small World*, HNSW) [5]. Egzaktni indeks poredi upit sa svakim vektorom u kolekciji i garantuje tačan rezultat, uz cenu linearne složenosti po broju vektora. Aproksimativni indeks žrtvuje malu meru tačnosti zarad znatno veće brzine, što je neophodno tek pri pretrazi nad milionima ili milijardama vektora. Budući da korpus u ovom radu obuhvata svega nekoliko hiljada segmenata, odabran je egzaktni indeks, čime se uklanja aproksimacija kao mogući izvor greške i obezbeđuje da izmerene razlike između strategija potiču isključivo od strategije pretrage, a ne od indeksne strukture.

Model koji je korišćen pripada savremenoj liniji kontekstualnih ugradnji, koja se razvila iz ranijih statičkih pristupa. Kod statičkih ugradnji, npr. Word2Vec, svaka reč dobija jedinstven vektor nezavisno od konteksta, pa višeznačne reči imaju samo jednu reprezentaciju. Pojavom arhitekture transformera i modela tipa BERT [10] uvedene su kontekstualne ugradnje, kod kojih reprezentacija reči zavisi od rečenice u kojoj se javlja. Model text-embedding-3-small nastavak je te linije i proizvodi ugradnju celog odsečka teksta koja objedinjuje doprinose svih tokena u njihovom međusobnom kontekstu.

```
from langchain_community.vectorstores import FAISS
def build_vector_store(docs: List[Document]) -> FAISS:
    logger.info("Izgradnja FAISS vektorski indeks...")
    store = FAISS.from_documents(docs, get_embeddings())
    logger.info("FAISS gotov.")
    return store
```

Slika 3: Izgradnja FAISS indeksa i pretraga (iz tool_factory.py).

2.3.3 Hibridna pretraga i recipročna fuzija rangova

Hibridna pretraga objedinjuje leksičku i vektorsku strategiju kako bi iskoristila njihove komplementarne prednosti: BM25 hvata egzaktna poklapanja, a vektorska pretraga semantičke veze. Jedan od najrobusnijih i najjednostavnijih postupaka spajanja rezultata je recipročna fuzija rangova (RRF) . RRF dodeljuje dokumentu skor obrnuto srazmeran njegovom rang u

svakoj od lista, čime se izbegava potreba za usklađivanjem nesamerljivih skorova različitih sistema:[6]

$$RRF(d) = \sum 1 / (k + r_r(d)) \quad (12)$$

Recipročna fuzija rangova ima dva svojstva koja je čine pogodnom za kombinovanje heterogenih pretraživača. Prvo, doprinos pojedinačne liste opada monotono sa rangom dokumenta, jer je funkcija $1/(k + r)$ strogo opadajuća po r , pa bolje rangirani dokumenti dobijaju veći doprinos. Drugo, metoda zavisi isključivo od rangova, a ne od apsolutnih vrednosti skorova pojedinačnih pretraživača, pa ne zahteva normalizaciju međusobno neuporedivih skorova ([6] npr, neograničenih BM25 skorova i kosinusne sličnosti iz opsega $[-1,1]$). Konstanta k ublažava uticaj najviše rangiranih dokumenata; pri većim vrednostima k razlike između prvih rangova postaju manje izražene. U ovom radu korišćena je vrednost $k = 60$, uobičajena u literaturi. Rang dokumenta d u rang-listi je $r_r(d)$, a k konstanta koja ublažava uticaj veoma visokih rangova (kanonska vrednost je $k = 60$). U konkretnoj implementaciji primenjena je varijanta zasnovana na recipročnoj funkciji ranga ($1/(\text{rang}+1)$) sa ponderom α koji uravnotežuje doprinos vektorske i leksičke liste; pri $\alpha = 0,5$ obe strategije imaju jednaku težinu. Detaljna analiza ove implementacione odluke data je u poglavlju 4.

```
def _hybrid(query, vector_store, bm25_retriever):
    vec_docs = vector_store.similarity_search(query, k=TOP_K_VECTOR)
    bm25_docs = bm25_retriever.get_relevant_documents(query)
    scores = {}
    for rank, doc in enumerate(vec_docs):
        key = doc.page_content[:200]
        scores.setdefault(key, {"doc": doc, "score": 0.0})
        scores[key]["score"] += HYBRID_ALPHA * (1.0 / (rank + 1))
    for rank, doc in enumerate(bm25_docs):
        key = doc.page_content[:200]
        scores.setdefault(key, {"doc": doc, "score": 0.0})
        scores[key]["score"] += (1 - HYBRID_ALPHA) * (1.0 / (rank + 1))
    return [v["doc"] for v in sorted(scores.values(), key=lambda x: x["score"], reverse=True)[:TOP_K_HYBRID]]
```

Slika 4: Implementacija hibridne fuzije (iz `tool_factory.py`).

2.3.3.1 Detaljna analiza fuzije rangova

Recipročna fuzija rangova zasniva se na jednostavnoj ideji: umesto da se kombinuju sirovi skorovi različitih sistema (koji su međusobno nemerljivi, BM25 vraća neograničene pozitivne skorove, dok kosinusna sličnost vraća vrednosti u opsegu $[-1,1]$), kombinuju se rangovi, koji su uvek u istoj, uporedivoj skali. Dokument koji je rangiran prvi u nekoj listi dobija doprinos $1/(k+1)$, drugi $1/(k+2)$ i tako dalje, bez obzira na to koliki je bio njegov sirovi skor. Time se izbegava potreba za normalizacijom skorova, koja je osetljiva i sklona greškama.

Primer izračunavanja hibridnog skora za $\alpha = 0,5$ prikazan je u Tabeli 2.3. Pretpostavimo da za neko pitanje vektorska lista vraća dokumente [A, B, C], a BM25 lista [B, D, A]. Doprinosi se akumuliraju po dokumentu:

Tabela 2: Brojčani primer hibridne fuzije rangova ($\alpha = 0,5$).

Dokument	Vektorski doprinos	BM25 doprinos	Ukupan skor
A	$0,5 \cdot (1/1) = 0,500$	$0,5 \cdot (1/3) = 0,167$	0,667
B	$0,5 \cdot (1/2) = 0,250$	$0,5 \cdot (1/1) = 0,500$	0,750
C	$0,5 \cdot (1/3) = 0,167$	,	0,167
D	,	$0,5 \cdot (1/2) = 0,250$	0,250

Rezultujući poredak je $B (0,750) > A (0,667) > D (0,250) > C (0,167)$. Primer pokazuje da dokument B, koji je u obe liste rangiran visoko, izbija na prvo mesto iako ni u jednoj pojedinačnoj listi nije bio ubedljivo prvi, to je suština fuzije: konsenzus dva nezavisna signala je pouzdaniji od bilo kog pojedinačnog signala. Ovaj mehanizam objašnjava zašto hibridna strategija u eksperimentu retko potpuno promašuje: da bi neki relevantan segment bio izgubljen, moralo bi da ga propuste obe strategije istovremeno, što je znatno ređe nego da ga propusti jedna.

Implementaciona varijanta korišćena koristi $1/(\text{rang}+1)$ umesto kanonskog $1/(k+\text{rang})$ sa $k=60$. Razlika je neznatna ali bitna: pri malom efektivnom k , razlike između prvih nekoliko rangova su izraženije (prvi dobija 0,5, drugi 0,33, treći 0,25), čime se snažnije nagrađuju vrhunski rangirani dokumenti. Pri velikom k (kao u kanonskoj verziji), kriva doprinosa je položenija i razlike između rangova su ublažene. Za male liste ($K=5$), kako je slučaj ovde, izraženija varijanta je razumna jer je relevantan segment gotovo uvek među prva dva-tri rezultata.

U implementaciji se kao ključ za prepoznavanje istog segmenta u obe liste koristi prvih 200 znakova njegovog sadržaja (`doc.page_content[:200]`). Pošto isti segment u vektorskoj i u leksičkoj listi ima identičan početni tekst, ovaj skraćeni isečak služi kao jeftin i pouzdan identifikator pri sabiranju doprinosa rangova, bez potrebe za poređenjem celokupnog sadržaja segmenta.

2.3.3.2 Teorijsko utemeljenje recipročne fuzije rangova

Ključno svojstvo metode jeste to što kombinuje rangove, a ne skorove pojedinačnih pretraživača. Skorovi koje vraćaju BM25 i vektorska pretraga međusobno su nemerljivi: BM25

vraća neograničene pozitivne vrednosti čija skala zavisi od učestalosti termina i veličine kolekcije, dok kosinusna sličnost pripada ograničenom opsegu $[-1,1]$. Sabiranje ili usrednjavanje takvih vrednosti bilo bi besmisleno bez prethodne normalizacije, a svaka normalizacija uvodi dodatne pretpostavke o raspodeli skorova. Korišćenjem ranga, koji je samo redni broj dokumenta u listi, recipročna fuzija u potpunosti zaobilazi problem nesamerljivosti skala [6].

Uloga konstante k u imeniocu je stabilizaciona, bez nje, dokument rangiran na prvom mestu dobio bi doprinos jedan, a već drugorangirani upola manji, čime bi najviši rangovi nesrazmerno dominirali konačnim poretom. Dodavanjem konstante k , doprinosi najviših rangova se ublažavaju i međusobno približavaju, pa fuzija postaje otpornija na slučajne razlike u vrhu pojedinačnih listi. Empirijska analiza [6] pokazala je da vrednost $k = 60$ daje stabilne rezultate na širokom skupu kolekcija, te je ta vrednost postala uobičajeni izbor i primenjena je i u ovom radu.

U poređenju sa drugim metodama agregacije rang-listi, recipročna fuzija pokazuje praktične prednosti. Metoda linearne kombinacije (engl. *linear combination*) sabira normalizovane skorove uz težinske koeficijente, ali zahteva izbor tih koeficijenata i osetljiva je na postupak normalizacije. Bordin metod (engl. *Borda count*) dodeljuje poene obrnuto srazmerno rangu na linearnoj skali, čime ne ublažava dominaciju najviših rangova u meri u kojoj to čini recipročna fuzija. Recipročna fuzija objedinjuje jednostavnost Bordinog metoda sa otpornošću linearne kombinacije, bez potrebe za podešavanjem težina ili normalizacijom.

Recipročna fuzija dostiže određenu teorijsku garanciju. Pošto se oslanja isključivo na rangove, ona uspeva da agregira signale i kada su skale pojedinačnih pretraživača potpuno nesamerljive, jer relativni poredak dokumenata ostaje smislen bez obzira na apsolutne vrednosti. Ovo svojstvo povezuje recipročnu fuziju sa teorijom glasanja, u kojoj se traži kolektivni poredak na osnovu pojedinačnih rangiranja. Za razliku od prostog zbira glasova, koji može dovesti do Kondorseovog paradoksa (engl. *Condorcet paradox*), gde kolektivni poredak postaje protivrečan iako su pojedinačni poreci konzistentni, recipročna fuzija u praksi daje stabilan i konzistentan poredak, što je čini pogodnom za spajanje heterogenih pretraživača.

2.3.5 Ponovno rangiranje i napredne tehnike

Pored tri strategije koje su predmet ovog eksperimenta, savremeni RAG sistemi često uvode i dodatni korak ponovnog rangiranja (engl. *re-ranking*). Ideja je da se najpre, jeftinom i brzom

pretragom, izdvoji veći skup kandidata (npr. top-50), a zatim da se taj skup precizno preuredi skupljim, ali tačnijim modelom, najčešće unakrsnim koderom (engl. *cross-encoder*), koji zajedno kodira par (upit, segment) i daje precizniju ocenu relevantnosti od pukog poređenja nezavisnih ugradnji.

Unakrsni koder se razlikuje od dvojnog koder (engl. *bi-encoder*) koji se koristi u vektorskoj pretrazi. Dvojni koder nezavisno ugrađuje upit i segment, pa njihovu sličnost računa naknadno, što je brzo (ugradnje se mogu unapred izračunati), ali manje precizno. Unakrsni koder propušta upit i segment kroz model zajedno, omogućavajući im da „međusobno obrate pažnju“, što daje tačniju ocenu po cenu mnogo većeg računskog troška (ne mogu se unapred izračunati). Zbog toga se u praksi često koristi dvostepeni pristup: brz dvojni koder za grubo izdvajanje, a zatim tačan unakrsni koder za fino rangiranje malog skupa kandidata.

U ovom radu ponovno rangiranje nije primenjeno, iz dva razloga. Prvo, cilj rada je čisto poređenje osnovnih strategija pretrage, a uvođenje rerankera unelo bi dodatnu promenljivu koja bi zamaglila to poređenje. Drugo, hibridna fuzija već obavlja oblik preuređivanja na osnovu konsenzusa dva signala, što delom ostvaruje sličan cilj uz znatno manju složenost. Ipak, dodavanje rerankera čini prirodan i obećavajući pravac proširenja.

2.4 Agentski sistemi i ReAct paradigma

Agent je softverski entitet koji opaža okruženje, donosi odluke i preduzima akcije radi ostvarenja cilja. U kontekstu jezičkih modela, agent je sistem u kome model ne samo da generiše tekst, već i odlučuje o pozivima spoljašnjih alata (pretraga, kalkulator, API). Razlikuju se reaktivni agenti (reaguju neposredno na ulaz), deliberativni agenti (planiraju unapred), agenti zasnovani na pozivanju alata i višeagentski sistemi. Sistem u ovom radu je agent zasnovan na alatima, pri čemu su alati tri strategije pretrage.

ReAct [2], prepliće zaključivanje (engl. *reasoning*) i delovanje (engl. *acting*) u jedinstvenoj petlji. Model naizmenično generiše misli (Thought), akcije (Action) i posmatra ishode (Observation), sve dok ne dođe do konačnog odgovora (Final Answer). Time se eksplicitno modeluje proces rezonovanja i omogućava iterativno, kontekstom vođeno pozivanje alata.

```
Request: POST https://aoai-farm.bosch-temp.com/api/openai/deployments/askbosch-prod-farm-openai-gpt-4o-mini-2024-07-18/d
at/completions?api-version=2024-08-01-preview "HTTP/1.1 200 OK"
I now have enough information to answer.
Final Answer: Each cycle in Fig. 14.10 lasts a tenth of a second, hence  $T = 0.1$  seconds.
A waveform that is the sum of two sine waveforms, one of frequency 10
Hz (note five repetitions in the half-second window) and one of frequency 100 Hz, both of
amplitude 1.
We can represent these two component frequencies with a spectrum. The spec-
trum of a signal is a representation of each of its frequency components and their
amplitudes. Figure 14.18 shows the spectrum of Fig. 14.17. Frequency in Hz is
```

Slika 4: Primer ReAct petlje.

Na osnovu opisane petlje misli, akcije i opažanja, ponašanje ReAct agenta može se formalno opisati kao proces odlučivanja u uslovima u kojima ishodi delom zavise od okruženja, odnosno kao Markovljev proces odlučivanja (engl. *Markov Decision Process*, MDP). Stanje s_t u koraku t obuhvata istoriju dotadašnjih misli, akcija i opažanja; akcija a_t je izbor alata sa pripadajućim argumentima ili odluka da se proces okonča konačnim odgovorom; opažanje o_t je rezultat koji okruženje, odnosno pozvani alat, vraća agentu. Politiku izbora akcije agent implicitno definiše kroz uslovnu raspodelu:

$$\pi(a_t | s_t) = p_{LLM}(a_t | s_t), \quad s_{t+1} = s_t \cup \{a_t, o_t\} \quad (12)$$

gde jezički model, na osnovu prompta koji kodira tekuće stanje, dodeljuje raspodelu verovatnoća nad mogućim narednim akcijama, a naredno stanje formira se dopunjavanjem istorije novom akcijom i pripadajućim opažanjem. Iako model nije eksplicitno treniran kao agent u okviru učenja uslovljavanjem, on tokom generisanja teksta efektivno uzorkuje akcije iz ove raspodele, pa se ReAct petlja može tumačiti kao sprovođenje naučene politike nad opisanim prostorom stanja.

Ova formalizacija razjašnjava razliku između ReAct paradigme i srodnih pristupa. Kod lanca misli (engl. *Chain-of-Thought*) [18], model generiše niz međukoraka rezonovanja, ali bez mogućnosti da deluje na okruženje; sva razmatranja odvijaju se nad parametarskim znanjem modela, pa pogrešna ili zastarela pretpostavka ne može biti ispravljena spoljašnjom proverom. Standardni pristup upotrebi alata, sa druge strane, poziva alat ali bez eksplicitnog rezonovanja o tome zašto i kada to čini. ReAct paradigma [2] objedinjuje oba pristupa naizmeničnim prepletanjem rezonovanja i delovanja: misao usmerava izbor akcije, a opažanje dobijeno akcijom koriguje naredno rezonovanje, čime se smanjuje oslanjanje na potencijalno netačno parametarsko znanje.

Izbor gornje granice broja iteracija nije proizvoljan, već predstavlja kompromis između potpunosti i cene. Svaka iteracija podrazumeva najmanje jedan poziv jezičkog modela i, po pravilu, jedan poziv alata, što povećava i vreme odziva i novčani trošak. Premali broj iteracija

može prekinuti rezonovanje pre nego što agent prikupi dovoljno konteksta, dok preveliki broj povećava trošak i rizik od ulaska u neproduktivne petlje. U ovom radu vrednost ograničenja postavljena je tako da omogući nekoliko koraka pretrage uz kontrolisan trošak, što se pokazalo dovoljnim za pitanja u test skupu.

Agentski RAG treba razlikovati od pristupa Self-RAG [9]. Kod Self-RAG-a, sam jezički model je dodatno treniran da tokom generisanja emituje posebne kontrolne oznake kojima odlučuje kada da pokrene pretragu i da kritički oceni sopstveni izlaz, pa je odlučivanje ugrađeno u model. Kod agentskog RAG-a, odlučivanje je izmešteno u spoljašnji agent koji bira između jasno razdvojenih strategija pretrage, bez izmene samog modela. Svesno je izabran agentski pristup jer omogućava izolovano poređenje strategija pod kontrolisanim uslovima, što kod Self-RAG-a, gde su strategije utopljene u parametre modela, ne bi bilo izvodljivo.

U biblioteci LangChain, ReAct agent se konstruiše funkcijom `create_react_agent`, a izvršava ga `AgentExecutor`, koji upravlja petljom poziva alata, ograničava broj iteracija i obrađuje greške parsiranja. Ključni parametri u ovom radu su maksimalan broj iteracija je šest, maksimalno vreme izvršavanja (120 sekundi) i mehanizam za obradu grešaka parsiranja, koji agentu vraća uputstvo o ispravnom formatu odgovora kada model odstupa od očekivane strukture.

```
executor = AgentExecutor(  
    agent=agent,  
    tools=tools,  
    verbose=True,  
    max_iterations=6,  
    max_execution_time=120,  
    handle_parsing_errors=handle_error,  
    return_intermediate_steps=True,  
    early_stopping_method="generate",  
)  
logger.info(f"Agent [{mode}] spreman sa alatima: {[t.name for t in tools]}")  
return executor
```

Slika 5: Konfiguracija `AgentExecutor`-a (iz `agent_orchestrator.py`).

ReAct paradigma povezuje dva ranija pristupa između dva ranija pristupa. Prvi pristup, čisto zaključivanje (npr. chain-of-thought), navodi model da glasno razmišlja korak po korak, ali nema pristup spoljašnjim izvorima, pa može da izvodi formalno ispravne zaključke iz pogrešnih ili izmišljenih činjenica. Drugi pristup, čisto delovanje, poziva alate ali bez eksplicitnog rezonovanja o tome zašto i kako, pa lako zaluta. ReAct ih spaja: misao (Thought)

usmerava sledeću akciju, a posmatranje (Observation) rezultata akcije uslovljava sledeću misao. Time se dobija petlja u kojoj se rezonovanje i prikupljanje dokaza uzajamno koriguju.

U kontekstu ovog rada, ReAct petlja ima i metodološku ulogu: ona čini proces odlučivanja agenta transparentnim i zapisivim. Svaki poziv alata i svaki vraćeni kontekst beleže se kao međukorak, što kasnije omogućava RAGAS evaluatoru da tačno zna koji je kontekst korišćen za generisanje odgovora. Bez ovog eksplicitnog traga, metrike odziva i preciznosti konteksta se ne bi mogle izračunati, jer one porede pronađeni kontekst sa referentnim odgovorom.

Agentski sloj uvodi i specifične rizike koji su se materijalizovali u eksperimentu. Pošto model mora da generiše izlaz u tačno propisanom tekstualnom formatu (Thought/Action/Action Input/Observation/Final Answer), svako odstupanje od tog formata izaziva grešku parsiranja. U zabeleženim rezultatima pojavile su se i poruke o prevremenom zaustavljanju agenta (povezane sa mehanizmom `early_stopping_method`), što je deo razloga za pojedine nedovršene odgovore. Ova zapažanja detaljno se analiziraju u poglavljima 6 i 7 i predstavljaju važan, dokumentovan nalaz o ceni agentske fleksibilnosti.

Ponašanje ReAct agenta u velikoj meri određuje šablon prompta koji mu nameće strukturu, mora da postigne nekoliko ciljeva istovremeno: da modelu objasni dostupne alate, da propiše tačan format izlaza (kako bi se mogao mašinski parsirati), da ograniči broj koraka (kako bi se sprečile beskonačne petlje) i da definiše šta da radi kada odgovor nije pronađen. Svaki od ovih zahteva odražava se u konkretnim delovima šablona koji su korišćeni.

Bez fraze „Final Answer:“ model bi mogao da nastavi da generiše misli i akcije unedogled ili da prevremeno prekine bez jasnog odgovora. Ograničenje na najviše dva koraka posmatranja čini kompromis: dovoljno da agent po potrebi izvrši jednu dopunsku pretragu, ali ne toliko da se izgubi u dugim, skupim i nestabilnim lancima. Kako rezultati pokazuju, čak i uz ova ograničenja, deo grešaka potiče upravo od odstupanja modela od propisanog formata, što je inherentan rizik tekstualno strukturisanih agenata.

U agentskom RAG sistemu, opisi alata (engl. *tool descriptions*) su aktivni mehanizam upravljanja ponašanjem agenta. Pošto ReAct agent bira alat na osnovu njegovog opisa, formulacija tih opisa direktno oblikuje odluke agenta. U ovom sistemu, opis hibridnog alata eksplicitno ga predstavlja kao „najbolji prvi izbor za svako pitanje“, čime se agent suptilno usmerava da podrazumevano poseže za fuzijom, a da specijalizovane alate (čisto leksički, čisto vektorski) koristi samo kada proceni da je to potrebno.

Ovakav način definisanja alata utiče na ponašanje agenta. U sistemima vođenim jezičkim modelima, prirodni jezik ima ulogu sloja konfiguracije, pa izmena jedne rečenice u opisu alata može promeniti ponašanje agenta u meri uporedivoj sa izmenom numeričkog parametra. Ta osobina istovremeno predstavlja izvor nestabilnosti, jer manje izmene u formulaciji mogu dovesti do nepredvidivih posledica, što agentske sisteme čini težim za pouzdano podešavanje od klasičnih, deterministički konfigurisanih cevovoda. Ovo zapažanje detaljnije se razmatra u poglavlju 7.

2.4.6 Agentski naspram standardnog RAG-a

Standardni i agentski RAG razlikuju se po nekoliko ključnih svojstava. Standardni RAG pronalaženje izvršava jednom i unapred, po fiksnoj strategiji, što ga čini jednostavnim, lako ponovljivim i sa visokom kontrolom, ali bez sposobnosti da odgovori na višekorne (engl. *multi-hop*) upite. Agentski RAG, nasuprot tome, pronalaženje sprovodi iterativno i po potrebi, prepuštajući agentu izbor alata; time dobija na fleksibilnosti i mogućnosti rešavanja složenijih upita, uz cenu veće složenosti i nešto niže ponovljivosti.

2.5 Evaluacija RAG sistema, RAGAS okvir

Tradicionalne metrike preklapanja, poput BLEU i ROUGE, mere površinsko poklapanje generisanog i referentnog teksta i slabo koreliraju sa stvarnim kvalitetom RAG odgovora, jer ne hvataju vernost kontekstu niti relevantnost. Savremeni pristup je „LLM kao sudija“ (engl. *LLM-as-judge*), u kome jak jezički model ocenjuje kvalitet odgovora prema preciznim kriterijumima. RAGAS [3] standardizuje ovaj pristup u skup ponovljivih, automatizovanih metrika posebno projektovanih za RAG.

Vernost (*faithfulness*) meri udeo tvrdnji u odgovoru koje su poduprte pronađenim kontekstom. Vrednost 1 znači da je svaka tvrdnja utemeljena, dok niže vrednosti ukazuju na halucinacije:

$$\begin{aligned} \text{Faithfulness} &= \frac{(\text{broj tvrdnji potvrđenih kontekstom})}{(\text{ukupan broj tvrdnji u odgovoru})} \end{aligned} \quad (13)$$

Relevantnost odgovora (*answer relevancy*) procenjuje koliko odgovor stvarno odgovara na postavljeno pitanje. Računa se tako što se na osnovu odgovora generiše više sintetičkih pitanja, a zatim se meri prosečna kosinusna sličnost njihovih ugradnji sa ugradnjom originalnog pitanja:

$$\text{AnswerRelevancy} = (1/N) \cdot \sum \cos(E(q_i), E(q)) \quad (14)$$

Odziv konteksta (*context recall*) meri koliko je tvrdnji iz referentnog (tačnog) odgovora pokriveno pronađenim kontekstom, to jest, da li je pretraživač pronašao sve što je potrebno:

$$\begin{aligned} \text{ContextRecall} & \\ &= (\text{tvrdnje iz reference pokrivenne kontekstom}) \\ & / (\text{ukupno tvrdnji u referenci}) \end{aligned} \tag{15}$$

Preciznost konteksta (*context precision*) meri odnos signala i šuma u pronađenim segmentima, da li su relevantni segmenti rangirani visoko:

$$\begin{aligned} \text{ContextPrecision} & \\ &= (\sum \text{Precision}@k \\ & \cdot v_k) / (\text{ukupno relevantnih segmenata u top} - K) \end{aligned} \tag{16}$$

gde je v_k binarni pokazatelj relevantnosti segmenta na poziciji k . Sve četiri metrike imaju opseg $[0,1]$, a više vrednosti znače bolji kvalitet.

Četiri RAGAS metrike nisu nezavisne; one mere komplementarne aspekte istog cevovoda i njihova kombinacija daje potpuniju sliku nego bilo koja pojedinačno. Mogu se grupisati u dve osnovne grupe. Prva grupa: odziv konteksta i preciznost konteksta zajedno ocenjuju da li je pretraživač obavio svoj posao: odziv meri da li je sve potrebno pronađeno, a preciznost da li je pronađeno bez previše šuma. Druga grupa tiče se generisanja: vernost meri da li je odgovor utemeljen u pronađenom kontekstu (odsustvo halucinacija), a relevantnost odgovora da li odgovor uopšte pogađa suštinu pitanja.

Ovaj raspored olakšava dijagnostičko tumačenje. Nizak odziv konteksta uz visoku vernost ukazuje da model pošteno priznaje neznanje kada kontekst nije pronađen (ne halucinira, ali ni ne odgovara). Visok odziv uz nisku vernost ukazuje na suprotan problem: kontekst je pronađen, ali ga model ignoriše i izmišlja. Visoka relevantnost odgovora uz nisku vernost znači da odgovor zvuči kao da pogađa pitanje, ali nije potkrepljen izvorom - najopasniji oblik halucinacije. U poglavlju 6, ovakvo dijagnostičko čitanje primenjeno je na konkretne slučajeve iz test skupa.

Na ovom test skupu odziv konteksta i preciznost konteksta imaju identične vrednosti po svakom pitanju. Razlog je priroda sintetičkih pitanja - svako pitanje generisano je iz tačno jednog izvornog segmenta, pa je relevantan tačno jedan segment. U takvom binarnom režimu (relevantan segment je ili pronađen ili nije), obe metrike se svode na isti binarni ishod. Ova pojava je sama po sebi koristan nalaz: ona ukazuje da bi za finije razlikovanje preciznosti i

odziva bio potreban test skup sa pitanjima koja zahtevaju više izvornih segmenata (višekorna pitanja).

Metrika vernosti je centralna za hipotezu H1. Postupak se odvija u dva koraka. U prvom koraku, model-sudija razlaže generisani odgovor a na skup pojedinačnih atomskih tvrdnji $\{c_1, c_2, \dots, c_m\}$. U drugom koraku, za svaku tvrdnju c_i proverava se da li je ona logički izvodljiva iz pronađenog konteksta C , dajući binarnu oznaku $v_i \in \{0,1\}$. Vernost je tada udeo potvrđenih tvrdnji:

$$Faithfulness(a, C) = (1/m) \sum_i v_i, \quad v_i = 1 \Leftrightarrow C \models c_i \quad (17)$$

Ovaj postupak objašnjava zašto se u rezultatima pojavljuju i nepotpune vrednosti poput 0,5 ili 0,667 (videti studije slučaja u poglavlju 6): one nastaju kada je deo tvrdnji u odgovoru poduprt kontekstom, a deo nije. Vrednost 0,5 znači da je tačno polovina atomskih tvrdnji utemeljena; vrednost 0,667 da su dve od tri tvrdnje utemeljene. Granularnost metrike na nivou tvrdnji čini je osetljivijom i informativnijom od proste binarne ocene tačno/netačno, te je iz tog razloga odabrana kao primarna mera sklonosti halucinacijama.

Analogno, relevantnost odgovora formalizuje se preko generisanja n sintetičkih pitanja $\{\hat{q}_1, \dots, \hat{q}_n\}$ iz odgovora a i merenja prosečne kosinusne sličnosti njihovih ugradnji sa ugradnjom originalnog pitanja q . Ako je odgovor dobro usmeren ka pitanju, rekonstruisana pitanja biće mu bliska, pa će sličnost biti visoka; ako odgovor luta ili je nepovezan, sličnost opada. Ova konstrukcija objašnjava zašto čak i činjenično tačan, ali preopširan odgovor može dobiti umanjenu relevantnost, jer uvodi sadržaj koji se ne preslikava nazad u izvorno pitanje.

Pošto za specifičan dokument retko postoji ručno anotiran skup pitanja sa tačnim odgovorima, u ovom radu test skup je generisan sintetički: jezičkom modelu se prosleđuje nasumično odabran segment dokumenta i traži se da formuliše jedno jasno pitanje i tačan odgovor (engl. *ground truth*) izvučen iz tog segmenta. Ovaj pristup je praktičan i skalabilan, ali nosi rizik cirkularnosti (isti tip modela generiše i ocenjuje), o čemu se detaljnije raspravlja u poglavlju 7.

```

prompt = f"""You are a question generation assistant. Based on the text below,
generate ONE clear and specific question that can be answered from the text, along with the correct answer.

Respond ONLY with valid JSON in this exact format (no extra text, no markdown):
{{"question": "...", "ground_truth": "..."}}

Rules:
- The question must be answerable from the provided text only.
- The ground_truth must be a concise, factual answer in English.
- Do not add any explanation outside the JSON.

Text:
{doc.page_content[:800]}
"""

```

Slika 6: Prompt za generisanje sintetičkog test skupa (iz *data_preparer.py*).

Tradicionalne metrike preklapanja niski znakova, kao što su BLEU i ROUGE, mere površinsko poklapanje generisanog i referentnog teksta. Takve metrike slabo su pogodne za vrednovanje RAG sistema jer odgovor može biti tačan i potpuno utemeljen u kontekstu, a da pritom koristi sasvim drugačiji izbor reči od reference, čime dobija nisku ocenu preklapanja. Iz tog razloga vrednovanje RAG sistema zahteva metrike koje procenjuju značenje, a ne doslovno poklapanje.

Savremeni pristup tom problemu jeste upotreba jezičkog modela u ulozi ocenjivača (engl. *LLM-as-judge*), na kojoj se zasniva okvir RAGAS [3]. Polazna pretpostavka je da dovoljno sposoban jezički model može da proceni semantičke odnose između pitanja, konteksta i odgovora pouzdanije od površinskih metrika. Meta-evaluacione studije pokazale su da ocene ovakvih modela u značajnoj meri koreliraju sa ljudskim procenama, što opravdava njihovu upotrebu kao približne zamene za skupo i sporo ljudsko ocenjivanje, uz svest o ograničenjima koja se razmatraju na kraju ovog potpoglavlja.

Metrika vernosti svodi se na problem prirodnog jezičkog zaključivanja (engl. *Natural Language Inference*, NLI). Odgovor se najpre razlaže na pojedinačne tvrdnje, a zatim se za svaku tvrdnju proverava da li je ona posledica (engl. *entailment*) pronađenog konteksta, da li mu protivreči (engl. *contradiction*) ili je u odnosu na njega neutralna. Vernost je tada udeo tvrdnji koje kontekst potkrepljuje, čime se izračunavanje vernosti svodi na niz NLI klasifikacija.

Metrika relevantnosti odgovora procenjuje koliko je odgovor usmeren upravo na postavljeno pitanje, a ne na srodne ali nepostavljene teme. Postupak se zasniva na obrnutom rezonovanju: iz datog odgovora jezički model generiše više sintetičkih pitanja na koja bi taj odgovor bio prirodan, a zatim se meri kosinusna sličnost između ugradnji tih pitanja i ugradnje stvarnog pitanja. Generisanje više pitanja, umesto jednog, smanjuje uticaj slučajnosti u pojedinačnom

generisanju i daje stabilniju ocenu; visoka prosečna sličnost znači da odgovor zaista odgovara na postavljeno pitanje.

Metrika odziva konteksta meri u kojoj meri pronađeni kontekst pokriva informacije sadržane u referentnom odgovoru. Referentni odgovor se razlaže na pojedinačne tvrdnje, pa se za svaku proverava da li je podržana pronađenim kontekstom; odziv je udeo pokrivenih tvrdnji. Ova metrika je neposredna analogija odziva (engl. recall) iz klasičnog pronalaženja informacija, prilagođena nivou tvrdnji umesto nivou dokumenata.

Metrika preciznosti konteksta procenjuje da li su relevantni segmenti rangirani visoko u listi pronađenih segmenata. Za razliku od proste preciznosti na preseku (engl. precision@k), koja ne uzima u obzir poredak, preciznost konteksta težinski vrednuje relevantne segmente prema njihovom rangi, pa segment koji je relevantan a rangiran visoko doprinosi više od jednako relevantnog segmenta rangiranog nisko. Time se nagrađuju strategije koje relevantan sadržaj postavljaju na vrh liste, što je u RAG sistemima važno jer model najveću pažnju posvećuje segmentima na početku konteksta.

Ograničenje ovakvog vrednovanja, poznato kao problem cirkularne evaluacije: isti jezički model učestvuje i u generisanju sintetičkog test skupa i u ocenjivanju odgovora. Usled toga, ocene mogu odražavati unutrašnju doslednost modela pre nego objektivni kvalitet sistema, jer model favorizuje formulacije bliske sopstvenim. Ova okolnost ne poništava vrednost merenja, budući da se sve strategije ocenjuju pod istim uslovima, pa su međusobno uporedive, ali nalaže opreznost pri tumačenju apsolutnih vrednosti metrika, što se detaljnije razmatra u poglavlju o rezultatima i diskusiji.

3. PREGLED LITERATURE

U ovom poglavlju sistematizuju se ključni radovi koji čine teorijsku i metodološku osnovu istraživanja, grupisani u tri tematske celine: razvoj RAG paradigme, strategije pronalaženja i agentski sloj sa evaluacijom. Nakon toga se rad pozicionira u odnosu na postojeću literaturu. Svi izvori navedeni su prema brojčanom sistemu i dati u poglavlju Literatura.

3.1 Razvoj i generacije RAG paradigme

Razvoj proširenog generisanja pretragom prošao je kroz nekoliko faza. Paradigma je uvedena u radu [1], gde je predloženo kombinovanje parametarskog znanja jezičkog modela sa neparametarskim znanjem iz spoljašnje baze podataka i pokazano da takvi modeli postižu bolje rezultate na zadacima intenzivnim po znanju, uz specifičnije i činjenično utemeljenije odgovore. Sistematizaciju oblasti dao je pregledni rad [7], u kome je uvedena podela na naivni, napredni i modularni RAG, prihvaćena i u ovom radu. Dalji razvoj išao je u dva pravca: u radu [9] predložen je Self-RAG, koji uči da sam odlučuje kada treba da pronalazi informacije i da kritički procenjuje sopstvene izlaze, dok grafovski pristup iz rada [15] uvodi globalno sumiranje nad grafom znanja izvedenim iz korpusa. Ova dva pravca označavaju prelaz od statičkog ka dinamičkom i agentskom RAG-u, kome pripada i sistem razvijen u ovom radu.[1]

3.2 Strategije pronalaženja i njihova fuzija

Pronalaženje informacija oslanja se na dve komplementarne grupe metoda. Verovatnosni okvir relevantnosti i funkcija BM25 detaljno su izloženi u radu [4], i ta funkcija i danas čini referentnu leksičku osnovicu. Kritički osvrt iz rada [14] upozorava da se prednost neuronskih metoda često precenjuje kada se one porede sa nedovoljno podešenim osnovicama, što se pokazalo neposredno relevantnim za rezultate ovog rada. Sa druge strane, gusto pronalaženje pasusa uvedeno je u radu [8], gde je pokazano da gusti vektori mogu nadmašiti leksičke metode na otvorenom odgovaranju na pitanja, a praktičnu izvodljivost takve pretrage nad velikim skupovima vektora obezbeđuje biblioteka FAISS iz rada [5]. Spajanje dveju grupa metoda razmatrano je u radu [6], u kome je pokazano da recipročna fuzija rangova nadmašuje složenije sheme kombinovanja, dok studija iz rada [12] potvrđuje komplementarnost leksičkih i gustih metoda. Ti nalazi čine glavni teorijski oslonac za izbor hibridne strategije u ovom radu.[7][9][15]

3.3 Agentski sloj i evaluacija

Agentski sloj i evaluacija predmet su novije literature. ReAct paradigma, uvedena u radu [2], povezuje zaključivanje i delovanje u jedinstvenu petlju i čini osnovu agentskog sloja ovog sistema, dok je u radu [11] predstavljen Toolformer, model koji sam uči da koristi spoljašnje alate. U pogledu vrednovanja kvaliteta, okvir RAGAS iz rada [3] uvodi metrike vernosti, relevantnosti odgovora, odziva i preciznosti konteksta, koje se koriste i u ovom radu, dok komplementaran okvir ARES iz rada [16] potvrđuje da je automatska evaluacija RAG sistema postala zasebno polje istraživanja. Najzad, model ugradnji korišćen u radu pripada liniji opisanoj u radu [13], u kome je predstavljeno treniranje tekstualnih ugradnji kontrastnim predtreniranjem.[4][14]

3.4 Pozicioniranje rada u odnosu na literaturu

Pregledana literatura otkriva nekoliko jasnih tokova istraživanja: formalizaciju RAG paradigme [1], taksonomiju RAG sistema [7], jačanje pojedinačnih strategija pretrage (leksičke [4] i guste [8]), kombinovanje strategija [6][12], agentizaciju [2][11][9] i automatizaciju evaluacije [3]. Svaki od ovih tokova bavi se jednim delom problema, ali su retki radovi koji ih objedinjuju u jedinstven, ponovljiv eksperiment nad istim korpusom i istim test skupom.

U odnosu na navedene radove, ovaj rad se ne bavi uvođenjem novog algoritma pretrage niti nove metrike; njegov doprinos je integrativan i empirijski. Sve tri strategije postavljaju se pod identične uslove (isti dokument, isti test skup, isti model, ista temperatura i isti top-K), čime se izoluje jedina nezavisna promenljiva, to jest strategija pronalaženja, i omogućava njihovo poređenje. Rezultati se tumače kritički, u skladu sa upozorenjem iznetim u radu [14] o potcenjivanju leksičkih osnovica, čime se izbegava pristrasnost u korist neuronskih metoda.

Tabela 3 sažeto pozicionira ključne radove prema tome koji aspekt sistema iz ovog rada potkrepljuju

Tabela 3: Pozicioniranje rada prema postojećoj literaturi

Tematska celina	Ključni izvori	Veza sa ovim radom
RAG paradigma	[1], [7]	Konceptualni okvir, taksonomija
Leksička pretraga	[4], [14]	BM25 osnovica, kritički stav
Vektorska pretraga	[5], [8]	FAISS, gusto pronalaženje
Hibridna fuzija	[6], [12]	RRF kao mehanizam fuzije

Tematska celina	Ključni izvori	Veza sa ovim radom
Agentski sloj	[2], [11], [9]	ReAct orkestracija alata
Evaluacija	[3], [16]	RAGAS i ARES metrike
Ugradnje	[13]	text-embedding-3-small

3.5 Detaljan osvrt na ključne radove

U nastavku se daje nešto detaljniji osvrt na radove koji su imali najveći uticaj na projektne i metodološke odluke, sa naglaskom na to šta je konkretno preuzeto. Brojevi u uglastim zagradama upućuju na odgovarajuću stavku u poglavlju Literatura.

[1] Pored uvođenja same paradigme, ovaj rad je uspostavio i terminološki okvir (retriever, generator, neparametarsko znanje) koji je usvojen u celom radu. Nalaz da pronalaženje poboljšava specifičnost i činjeničnost odgovora direktno motiviše centralnu tezu ovog rada o važnosti kvaliteta pretraživača.

[2] Rad o ReAct paradigmi pružio je ne samo konceptualnu osnovu agentskog sloja, već i konkretan obrazac prompta (Thought/Action/Observation) koji je, uz prilagođavanja, primenjen u modulu agent_orchestrator.py. Zapažanje da prepletanje rezonovanja i delovanja smanjuje halucinacije relevantno je za interpretaciju metrike vernosti.

[3] Rad o okviru RAGAS definisao je tačno one četiri metrike koje čine osnovu evaluacije u poglavlju 6. Posebno je preuzet pristup zasnovan na razlaganju odgovora na pojedinačne tvrdnje (za faithfulness) i na generisanju sintetičkih pitanja iz odgovora (za answer relevancy).

[6] Dokaz da jednostavna recipročna fuzija rangova nadmašuje složenije sheme kombinovanja bio je presudan za odluku da se u ovom radu primeni varijanta te tehnike umesto, recimo, učenja težina fuzije. Jednostavnost je ovde prednost, jer smanjuje broj hiperparametara i povećava ponovljivost.

[4] Iscrpan pregled verovatnosnog okvira relevantnosti pružio je teorijsko utemeljenje za tumačenje zašto BM25 dobro radi na terminološki bogatim korpusima, uvid koji je u poglavlju 6 potkrepljen empirijski.

[14] Zaključci tog rada korišćeni su kao osnova za tumačenje rezultata: prema neuronskim metodama pristupa se sa rezervom i one se porede sa dobro podešenim leksičkim osnovicama. Rezultati ovog rada, u kojima BM25 postiže visoke vrednosti, u skladu su sa tim stavom.

[5] Rad o biblioteci FAISS omogućio je praktičnu izvodljivost vektorske pretrage; iz njega je preuzet izbor egzaktnog indeksa za korpus umerene veličine, uz svest o postojanju aproksimativnih varijanti za veće razmere.

3.6 Komparativna analiza i pozicioniranje u odnosu na novije pravce

Na osnovu prethodnog pregleda, u ovom potpoglavlju rad se detaljnije pozicionira u odnosu na nekoliko aktuelnih istraživačkih pravaca, sa naglaskom na to po čemu se predloženo rešenje razlikuje od svakog od njih i zašto su određeni pristupi svesno izostavljeni.

3.6.1 Odnos prema izvornom RAG radu

Izvorni RAG rad [1] uveo je arhitekturu u kojoj se gusto pronalaženje pasusa (Dense Passage Retrieval) kombinuje sa generativnim modelom zasnovanim na arhitekturi BART, pri čemu se pronalazač i generator zajednički fino podešavaju na zadatku odgovaranja na pitanja. Sistem razvijen u ovom radu razlikuje se od te postavke u tri pogleda. Prvo, umesto isključivo guste pretrage, ovde se porede i kombinuju leksička i gusta strategija. Drugo, generator se ne fino podešava, već se koristi unapred obučeni model uz pristup zasnovan na podsticaju (engl. *prompting*), što odražava savremenu praksu. Treće, nad pronalaženjem stoji agentski sloj koji bira strategiju, čega u izvornom radu nema. Od pojave izvornog rada, oblast je napredovala ka modularnim i agentskim arhitekturama [7], čemu pripada i ovaj rad.

3.6.2 Self-RAG naspram agentskog RAG-a

Pristup Self-RAG [9] i agentski RAG predstavljaju dva različita načina uvođenja odlučivanja u proces pronalaženja. Kod Self-RAG-a, jezički model je dodatno treniran da sam, tokom generisanja, odlučuje kada je pronalaženje potrebno i da posebnim oznakama kritikuje sopstveni izlaz, pa je odlučivanje ugrađeno u parametre modela. Kod agentskog RAG-a, primenjenog u ovom radu, odlučivanje obavlja spoljašnji agent koji bira između jasno razdvojenih strategija, bez izmene modela. Self-RAG je pogodniji kada se raspolaze resursima za dodatno treniranje i kada je cilj potpuna autonomija modela, dok je agentski pristup pogodniji kada se traži modularnost, mogućnost zamene komponenti i, kao u ovom radu, kontrolisano poređenje strategija pod istim uslovima.

3.6.3 Grafovski RAG i razlozi za njegovo izostavljanje

Grafovski RAG [15] zamenjuje podelu na nezavisne segmente izgradnjom grafa znanja, u kome su entiteti čvorovi a njihovi odnosi grane, što omogućava globalno sumiranje i odgovaranje na pitanja koja zahtevaju povezivanje informacija razasutih po celom dokumentu. Takav pristup prevazilazi ograničenje podele na segmente, kod koje se veze između udaljenih delova teksta lako gube. Grafovski RAG bio bi pogodniji od ovde predloženog sistema za pitanja koja zahtevaju sintezu na nivou celog korpusa, na primer sažimanje glavnih tema knjige. U ovom radu grafovski pristup nije odabran jer uvodi znatnu složenost u izgradnji i održavanju grafa, a istraživačko pitanje rada usmereno je na poređenje leksičke, guste i hibridne pretrage nad istom podelom na segmente, gde bi uvođenje grafa narušilo uporedivost strategija.

3.6.4 Mesto rada u raspravi o leksičkom i neuronskom pronalaženju

Rad se smešta u širu raspravu o tome da li neuronske metode pronalaženja zaista nadmašuju klasične leksičke. Kritički osvrt [14] upozorava da se prednost neuronskih metoda često precenjuje usled poređenja sa slabo podešenim leksičkim osnovicama. Između čisto leksičkih i čisto gustih pristupa razvile su se i hibridne neuronske metode: model SPLADE [24] uči proređene leksičke reprezentacije proširene naučenim terminima, dok model ColBERT [25] uvodi pozno uparivanje (engl. *late interaction*) na nivou pojedinačnih tokena, čime spaja preciznost leksičkog uparivanja sa semantikom neuronskih reprezentacija. Ovaj rad ne uvodi takve modele, već empirijski poredi standardni BM25, gustu pretragu i njihovu fuziju; njegov doprinos toj raspravi jeste nalaz da na tehničkom korpusu dobro podešen BM25 ostaje konkurentan, što potkrepljuje upozorenje iz rada [14] konkretnim merenjima.

3.6.5 Ograničenja sintetičke evaluacije

Za vrednovanje RAG sistema razvijeno je više okvira. Pored okvira RAGAS [3] postoji i okvir ARES [16], koji se oslanja na klasifikatore dodatno obučene na malom skupu ljudski označenih primera, čime nastoji da smanji oslanjanje na ocene jednog jezičkog modela. Pored okvira za ocenjivanje, razvijeni su i standardizovani skupovi za proveru, poput skupa RGB [26], koji sistematski meri otpornost RAG sistema na šum. Ovaj rad ne koristi takav standardizovani skup, već sintetički test skup izveden iz samog korpusa, što je svesno ograničenje: ono obezbeđuje doslednost u odnosu na korišćeni dokument, ali umanjuje uporedivost sa rezultatima na javnim merilima i uvodi pristrasnost o kojoj se govori u poglavljima o rezultatima i diskusiji.

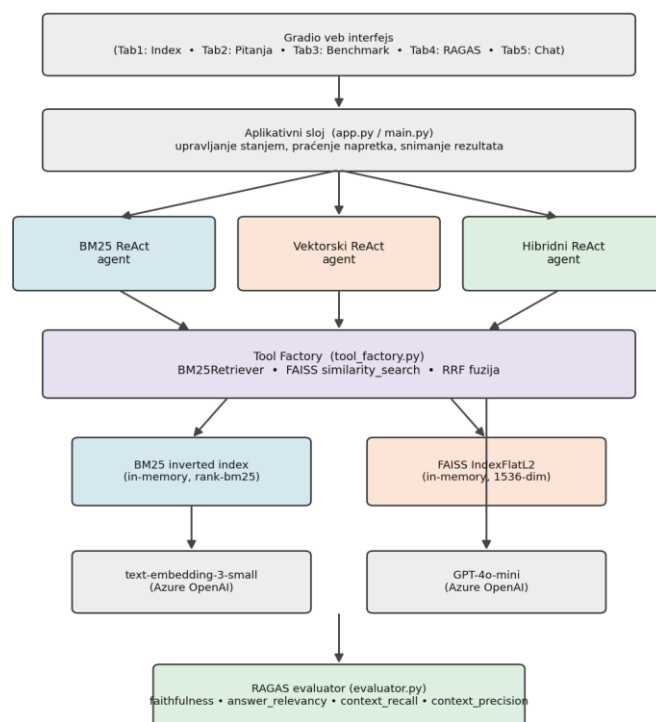
4. PROJEKTOVANJE SISTEMA

Ovo poglavlje opisuje arhitekturu sistema, tok podataka i ključne projektne odluke. Sistem je projektovan po modularnom načelu: svaka funkcionalna celina (priprema podataka, izgradnja indeksa, agentska orkestracija, evaluacija, korisnički interfejs) realizovana je kao zaseban modul sa jasno definisanim ugovorom.

4.1 Arhitektura sistema

4.1.1 Pregled komponenti

Sistem je organizovan u pet slojeva. Na vrhu se nalazi veb interfejs realizovan bibliotekom Gradio, sa pet kartica (indeksiranje, generisanje pitanja, benchmark, RAGAS evaluacija i interaktivni razgovor). Ispod njega je aplikativni sloj (app.py i main.py) koji upravlja stanjem i napretkom. Treći sloj čine tri ReAct agenta, leksički, vektorski i hibridni. Četvrti sloj je fabrika alata (tool_factory.py), koja izgrađuje pretraživače i implementira fuziju. Najniži sloj čine indeksi (BM25 i FAISS) zajedno sa modelima za ugradnju i generisanje. Slika 7 prikazuje ovu arhitekturu.

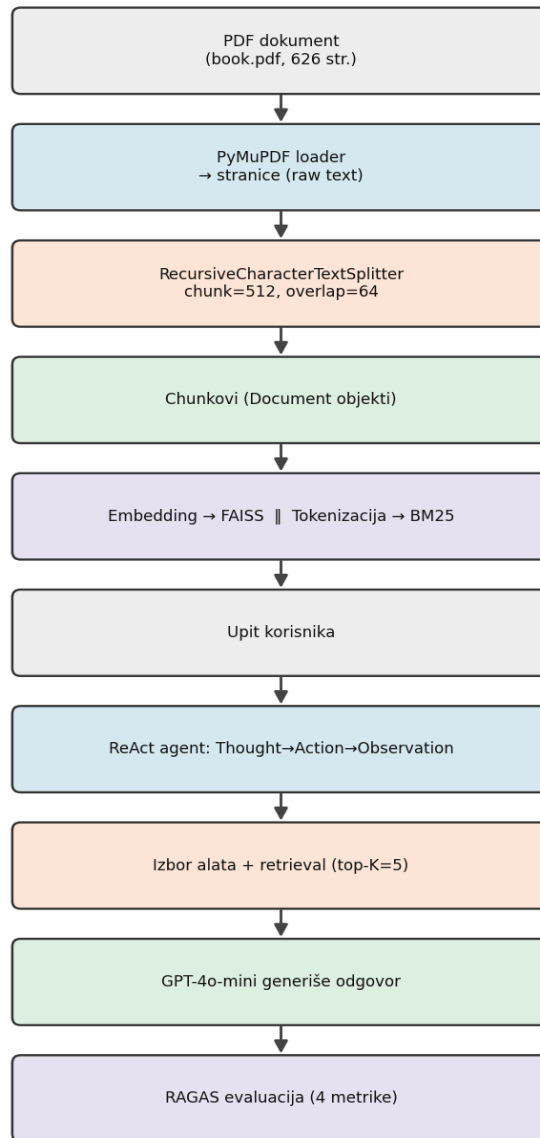


Slika 7: Slojevita arhitektura agentskog hibridnog RAG sistema.

Ključna projektna odluka je razdvajanje tri agenta sa različitim skupovima alata. Leksički agent ima na raspolaganju isključivo BM25 alat, vektorski agent isključivo FAISS alat, a hibridni agent sva tri alata (uključujući hibridni alat zasnovan na RRF fuziji). Ovakvo razdvajanje obezbeđuje čistu, kontrolisanu komparaciju strategija, jer se eliminiše mogućnost da agent „pobegne“ u drugu strategiju i time zamagli izvor uspeha ili neuspeha.

4.1.2 Tok podataka

Tok podataka kroz sistem može se podeliti na fazu pripreme (offline) i fazu upita (online). U fazi pripreme, PDF dokument se učitava pomoću PyMuPDF učitavača, deli na segmente rekurzivnim deliocem, a zatim se segmenti paralelno indeksiraju: vektorizuju se i smeštaju u FAISS indeks, te tokenizuju i smeštaju u BM25 invertovani indeks. U fazi upita, korisničko pitanje ulazi u ReAct agenta, koji bira alat, izvršava pretragu ($\text{top-K} = 5$), prosleđuje pronađeni kontekst modelu GPT-4o-mini, koji generiše odgovor; konačno, odgovor se prosleđuje RAGAS evaluatoru. Slika 4.2 prikazuje ovaj tok.



Slika 8: Tok podataka: od PDF dokumenta do RAGAS evaluacije.

4.1.3 Konfiguracija sistema

Svi parametri sistema centralizovani su u modulu config.py, čime se obezbeđuje ponovljivost i lako podešavanje. Tabela 4 navodi ključne parametre.

Tabela 4: Ključni konfiguracioni parametri sistema (config.py).

Parametar	Vrednost	Značenje
CHUNK_SIZE	512	Veličina segmenta u tokenima
CHUNK_OVERLAP	64	Preklapanje susednih segmenata

Parametar	Vrednost	Značenje
TOP_K_VECTOR	5	Broj segmenata iz vektorske pretrage
TOP_K_BM25	5	Broj segmenata iz BM25 pretrage
TOP_K_HYBRID	5	Broj segmenata posle fuzije
HYBRID_ALPHA	0,5	Pondera vektorske naspram leksičke liste
NUM_TEST_QUESTIONS	10 do 30	Broj generisanih test pitanja

4.2 Modul za pripremu podataka

Modul `data_preparer.py` obavlja dve funkcije: učitavanje i podelu dokumenta, pa generisanje sintetičkog test skupa. Učitavanje koristi `PyMuPDFLoader`, koji pouzdano izdvaja tekst uz očuvanje informacija o stranici, što je kasnije iskorišćeno za navođenje izvora u formatiranom kontekstu (npr. oznaka „[Izvor 1 | str.192 | book.pdf]“).

Generisanje test skupa zasniva se na nasumičnom uzorkovanju segmenata i traženju od modela da za svaki segment formuliše jedno pitanje i tačan odgovor u strogo definisanom JSON formatu. Temperatura modela pri generisanju pitanja postavljena je na 0,3 radi umerene raznovrsnosti, dok je pri odgovaranju i evaluaciji 0,0 radi determinizma.

Bitna projektna odluka je da su sva pitanja i odgovori na engleskom jeziku. Razlog je dvostruk: dokument je na engleskom, pa se izbegava gubitak u prevodu i metrike RAGAS-a, koje se oslanjaju na model-sudiju, pouzdanije rade na jeziku korpusa.

4.3 Modul za indeksiranje (fabrika alata)

Modul `tool_factory.py` izgrađuje FAISS i BM25 indekse i eksportuje tri fabričke funkcije: `create_tools` (hibridni agent sa sva tri alata), `create_vector_only_tools` (samo vektorski) i `create_bm25_only_tools` (samo leksički). Svaki alat je objekat klase `Tool` sa nazivom, izvršnom funkcijom i opisom; opis je važan jer ReAct agent na osnovu njega odlučuje koji alat da pozove. Pomoćna funkcija `_fmt` formatira pronađene segmente u čitljiv kontekst sa oznakama izvora i broja strane.

Umesto kanonskog RRF izraza sa konstantom $k = 60$, ovde je primenjena varijanta u kojoj se doprinos svakog dokumenta računa kao $\alpha \cdot 1/(\text{rang}+1)$ za vektorsku listu i $(1-\alpha) \cdot 1/(\text{rang}+1)$ za leksičku listu, uz $\alpha = 0,5$. Dokumenti se identifikuju po prvih 200 karaktera sadržaja, a skorovi se akumuliraju za dokumente koji se pojave u obe liste, što i jeste cilj fuzije: segment koji je istovremeno leksički i semantički relevantan dobija najviši ukupan skor. Ova varijanta je

matematički bliska RRF-u (oba pristupa koriste recipročnu funkciju ranga), ali uvodi eksplicitan pondera α , čime se omogućava fino podešavanje ravnoteže između strategija.

4.4 Agentski orkestrator

Modul `agent_orchestrator.py` konstruiše ReAct agenta i upravlja njegovim izvršavanjem. Centralni element je ReAct šablon prompta (`REACT_TEMPLATE`), koji modelu nameće strogu strukturu odgovora: `Thought` → `Action` → `Action Input` → `Observation` → `Final Answer`. Šablon eksplicitno ograničava broj koraka posmatranja na najviše dva i zahteva da agent uvek pruži konačan odgovor, čime se sprečavaju beskonačne petlje i nedovršeni izlazi.

Robusnost je obezbeđena na više nivoa. Funkcija `handle_error` vraća agentu uputstvo o ispravnom formatu kada model odstupa od strukture. Funkcija `run_agent` dodatno hvata slučajeve kada agent stane bez odgovora i, ako je moguće, izvlači odgovor iz poslednjeg posmatranog koraka. Sav pronađeni kontekst beleži se u međukoracima (`intermediate_steps`), što je neophodno za kasniju RAGAS evaluaciju, jer metrike odziva i preciznosti konteksta zahtevaju te pronađene segmente.

```
REACT_TEMPLATE = """Answer the following question using the available tools. You MUST follow this EXACT format every single time:

Thought: I need to search for information about this question.
Action: {tool_names_list}
Action Input: the search query
Observation: [tool result will appear here]
Thought: I now have enough information to answer.
Final Answer: [your answer based only on the documents]

STRICT RULES:
- You MUST use exactly one of these tool names: {tool_names}
- Action Input must be a plain string query, nothing else
- After at most 2 Observation steps, you MUST write "Final Answer:"
- If the documents don't contain the answer, write: Final Answer: The document does not contain information about this topic.
- Never say "Agent stopped" – always provide a Final Answer.

Tools available:
{tools}

Question: {input}

{agent_scratchpad}"""
```

Slika 9: ReAct šablon prompta (skraćeno, iz `agent_orchestrator.py`).

4.5 RAGAS evaluator

Modul `evaluator.py` sprovodi evaluaciju. Rezultati agenta (pitanje, odgovor, pronađeni kontekst, tačan odgovor) pretvaraju se u Hugging Face Dataset, nad kojim se pozivaju sve četiri metrike odjednom. Jezički model i model za ugradnju omotani su klasama `LangchainLLMWrapper` i `LangchainEmbeddingsWrapper`, čime se RAGAS-u prosleđuje isti

model (GPT-4o-mini, temperatura 0) koji se koristi i u ostatku sistema. Funkcija `compute_summary` računa prosek po metrici, ignorišući nedostajuće vrednosti.

Projektna odluka da se sve metrike izvrše odjednom (umesto metrika-po-metrika) doneta je radi jednostavnosti i robusnosti uz konkretnu verziju biblioteke RAGAS (0.1.13), čime se izbegavaju problemi sa ponovljenim inicijalizacijama i nekompatibilnostima asinhronog izvršavanja.

```
os.environ["NO_PROXY"] = "localhost,127.0.0.1,0.0.0.0"
os.environ["no_proxy"] = "localhost,127.0.0.1,0.0.0.0"

# — Proxy transport —————
def _make_http_client() -> httpx.Client:
    """Kreira httpx klijent sa proxy podešavanjima (kompatibilan sa httpx>=0.28)."""
    transport = httpx.HTTPTransport(proxy=HTTPS_PROXY, verify=False)
    return httpx.Client(transport=transport, verify=False, timeout=120.0)
```

Slika 10: Transport sa proksijem i izuzimanje localhost-a (iz `config.py`).

4.7 Projektni obrasci i načela

Sistem dosledno primenjuje nekoliko prepoznatljivih projektnih obrazaca. Obrazac fabrike (engl. *Factory*) realizovan je kroz tri funkcije koje proizvode skupove alata (`create_tools`, `create_vector_only_tools`, `create_bm25_only_tools`); time se konstrukcija agenata odvaja od njihove upotrebe i omogućava se da se ista orkestraciona logika primeni na različite konfiguracije alata. Obrazac strategije (engl. *Strategy*) ogleda se u tome što su tri pristupa pretrazi međusobno zamenljivi iza jedinstvenog interfejsa alata, pa agent može da bira među njima bez poznavanja njihove unutrašnje implementacije. Obrazac adaptera (engl. *Adapter*) primenjen je u evaluatoru, gde omotači `LangchainLLMWrapper` i `LangchainEmbeddingsWrapper` prilagođavaju modele interfejsu koji RAGAS očekuje.

Iznad pojedinačnih obrazaca, sistem počiva na načelu jasnog razdvajanja odgovornosti (engl. *separation of concerns*). Priprema podataka ne zna ništa o evaluaciji; fabrika alata ne zna ništa o korisničkom interfejsu; evaluator ne zna kako su alati implementirani. Ovakva razložena arhitektura olakšava testiranje, izmene i ponovnu upotrebu, a takođe je preduslov za kontrolisani eksperiment, jer omogućava da se zameni samo strategija pretrage, dok sve ostalo ostaje nepromenjeno.

4.7.1 Odluka o izolaciji strategija u zasebne agente

Sa stanovišta valjanosti eksperimenta, ključna projektna odluka jeste izolacija svake strategije u zaseban agent. Alternativno rešenje, u kojem bi jedan agent raspolagao svim alatima i sam ih birao, onemogućilo bi pripisivanje uspeha ili neuspeha konkretnoj strategiji, jer bi agent mogao da kombinuje alate na nepredvidiv način od pitanja do pitanja. Razdvajanjem na leksičkog, vektorskog i hibridnog agenta, od kojih svaki raspolaže strogo ograničenim skupom alata, postiže se da je strategija pretrage jedina kontrolisana razlika između tretmana. Ta odluka odgovara izolaciji nezavisne promenljive u klasičnom eksperimentu i predstavlja osnovu verodostojnosti rezultata iznetih u poglavlju 6.

4.7.2 Razmatranje skalabilnosti i ograničenja in-memory pristupa

Sistem je projektovan za in-memory rad: i FAISS indeks i BM25 indeks drže se u radnoj memoriji procesa. Za jedan dokument obima nekoliko hiljada segmenata, ovo je sasvim prikladno i donosi najmanju latenciju, bez potrebe za zasebnom bazom podataka. Međutim, za korpuse reda veličine miliona segmenata, ovaj pristup dostiže granice memorije i zahtevao bi prelazak na perzistentne vektorske baze (npr. sa diskovnim indeksima ili aproksimativnim pretragama poput HNSW) i na invertovane indekse koji se mogu memorijski mapirati. Ovo ograničenje je svesno prihvaćeno jer je u skladu sa obimom istraživanja, fokus je na poređenju strategija, a ne na inženjeringu velikih razmera, ali se eksplicitno navodi radi potpunosti i kao pravac mogućeg proširenja.

Latencija upita takođe zavisi od strategije. Leksička pretraga je najbrža jer radi nad invertovanim indeksom bez poziva spoljašnjim servisima. Vektorska pretraga uvodi mrežni poziv servisu za ugradnju upita pre same pretrage, što povećava latenciju i, kako se pokazalo u eksperimentu, uvodi tačku otkaza (greške u vezi). Hibridna pretraga izvršava obe operacije i potom fuziju, pa ima najveću latenciju, ali tu cenu plaća za najveću robusnost. Tabela 4.2 sažima ovaj kompromis.

4.8 Strategija obrade grešaka i otpornosti

Pošto je sistem sastavljen od više nepouzdatih komponenti (mrežni pozivi modelu, parsiranje izlaza agenta, spoljašnji servis za ugradnju), otpornost na greške projektovana je kao prvorazredna briga, a ne kao naknadna misao. Primenjeno je nekoliko slojeva odbrane. Na nivou poziva modela, prilagođeni HTTP klijent sa dovoljnim vremenskim ograničenjem (120

sekundi) sprečava neograničeno čekanje. Na nivou agenta, funkcija za obradu grešaka parsiranja vraća modelu jasnu instrukciju o ispravnom formatu umesto da sruši izvršavanje. Na nivou prikupljanja rezultata, sistem robusno hvata slučajeve kada agent stane bez konačnog odgovora i pokušava da odgovor rekonstruiše iz poslednjeg posmatranja.

Filozofija u osnovi ovih mehanizama je „neuspeh koji ne ruši sistem“ (engl. *graceful degradation*): bolje je vratiti delimičan ili pošteno priznat prazan rezultat nego srušiti celu evaluaciju zbog jednog problematičnog pitanja. Zahvaljujući ovakvom pristupu, eksperiment je uspeo da prikupi merenja za svih 30 pitanja po svakoj strategiji, uključujući i ona pitanja na kojima je neka strategija zakazala, što je omogućilo detaljan uvid u obrasce izostanaka. Deo zabeleženih grešaka, konkretno poruke o grešci u vezi pri pozivu servisa za ugradnju i poruke o nepodržanom načinu prevremenog zaustavljanja agenta, pokazuje da otpornost sistema nije bila potpuna. Te greške nisu izostavljene, već su zadržane u podacima i protumačene u poglavljima 6 i 7, jer čine deo realne slike o ponašanju sistema u radnim uslovima.

5. IMPLEMENTACIJA

Ovo poglavlje prikazuje implementaciju sistema po modulima, sa analizom izvornog koda i objašnjenjem ključnih odluka. Ceo sistem napisan je u programskom jeziku Python i organizovan u paket src sa modulima config, data_preparer, tool_factory, agent_orchestrator i evaluator, uz aplikativne fajlove app.py (Gradio) i main.py (CLI).

5.1 Tehnološki stek

Tabela 5 prikazuje korišćene tehnologije sa verzijama i obrazloženjem izbora. Verzije su pinirane u requirements.txt radi ponovljivosti.

Tabela 5: Tehnološki stek sa verzijama (requirements.txt).

Komponenta	Tehnologija	Verzija	Razlog izbora
Jezički model	GPT-4o-mini (Azure)	2024-07-18	Dostupan, ekonomičan, veliki kontekst
Ugradnje	text-embedding-3-small	v3	1536 dim., ekonomičan
Vektorska baza	FAISS CPU	$\geq 1.8.0$	In-memory, bez infrastrukture
Leksička pretraga	rank-bm25	$\geq 0.2.2$	Standardna implementacija BM25
Agentski okvir	LangChain	0.2.16	Zrela podrška za ReAct
Evaluacija	RAGAS	$\geq 0.1.13$	De-facto standard za RAG
Interfejs	Gradio	≥ 6.0	Brz interaktivni prototip
PDF parser	PyMuPDF	≥ 1.24	Pouzdan, čuva strukturu strane

5.2 Razvojno okruženje

Razvoj je obavljen u okruženju Visual Studio Code uz interpretator Python 3, u izolovanom virtuelnom okruženju (.venv). Pristup modelima ostvaren je preko korporativnog proksija. Izvorni kod je verzionisan i javno dostupan u repozitorijumu na platformi GitHub [20].

5.3 Implementacija po modulima

5.3.1 Priprema podataka

Funkcija `load_and_split_pdf` učitava PDF i deli ga na segmente. Funkcija `generate_test_questions` iterira kroz nasumično uzorkovane segmente, šalje model i parsira JSON odgovor, robusno preskačući segmente kod kojih parsiranje ne uspe. Svaki element test skupa sadrži pitanje, tačan odgovor i izvorni segment (engl. *context*).

```

for i, doc in enumerate(sample_docs[:n]):
    if progress_cb:
        progress_cb(i + 1, n, f"Generating question {i + 1}/{n}...")
    try:
        response = llm.invoke(prompt)
        raw = response.content.strip()
        raw = raw.replace("`json", "").replace("`", "").strip()
        parsed = json.loads(raw)
        if "question" in parsed and "ground_truth" in parsed:
            test_set.append({
                "question":    parsed["question"],
                "ground_truth": parsed["ground_truth"],
                "context":     doc.page_content,
            })
    except Exception as e:
        logger.warning(f"Skipping chunk {i}: {e}")

logger.info(f"Generated {len(test_set)} test questions.")
return test_set

```

Slika 11: Generisanje test pitanja (iz data_preparer.py).

5.3.2 Izgradnja indeksa i alata

Funkcije `build_vector_store` i `build_bm25_retriever` izgrađuju dva indeksa nad istim skupom segmenata. Fabričke funkcije zatim sklapaju alate. Najvažnija je `create_tools`, koja hibridnom agentu daje tri alata sa pažljivo formulisanim opisima, jer upravo te opise ReAct agent koristi pri odluci o pozivu. Opis hibridnog alata eksplicitno sugerise da je on „najbolji prvi izbor za svako pitanje“, čime se agent usmerava ka fuziji.

```

def create_tools(vector_store: FAISS, bm25_retriever: BM25Retriever) -> List[Tool]:
    """Hybrid agent - has all 3 retrieval strategies available."""
    return [
        Tool(name="hybrid_search",
            func=lambda q: _fmt(_hybrid(q, vector_store, bm25_retriever)),
            description="Hybrid search combining BM25 + FAISS using Reciprocal Rank Fusion. Best first choice for any question."),
        Tool(name="vector_search",
            func=lambda q: _fmt(vector_store.similarity_search(q, k=TOP_K_VECTOR)),
            description="Semantic vector search using FAISS embeddings. Good for conceptual questions and synonyms."),
        Tool(name="bm25_search",
            func=lambda q: _fmt(bm25_retriever.get_relevant_documents(q)),
            description="BM25 lexical keyword search. Good for exact names, terms, dates and numbers."),
    ]

```

Slika 12: Tri alata hibridnog agenta (iz tool_factory.py).

5.3.3 Orkestracija agenta

Funkcija `build_agent` kreira ReAct agenta i `AgentExecutor` sa parametrima opisanim u odeljku 4.4. Funkcija `run_agent` izvršava agenta nad jednim pitanjem, prikuplja konačan odgovor i sve pronađene kontekste, te robusno obrađuje greške i nedovršene izlaze. Povratna struktura sadrži odgovor, listu konteksta i međukorake, sve što je potrebno za evaluaciju.

```
contexts = []
for step in result.get("intermediate_steps", []):
    if len(step) >= 2 and isinstance(step[1], str) and len(step[1]) > 20:
        contexts.append(step[1])

return {
    "answer": answer,
    "contexts": contexts or ["Nije pronađen kontekst."],
    "steps": result.get("intermediate_steps", []),
}
```

Slika 13: Prikupljanje konteksta iz međukoraka (iz `agent_orchestrator.py`).

5.3.4 Evaluacija

Funkcija `run_ragas_evaluation` pretvara rezultate u Dataset i poziva RAGAS. Filter u `_build_dataset` osigurava da svaki red sadrži sve potrebne kolone (pitanje, odgovor, kontekst, tačan odgovor), čime se sprečavaju rušenja zbog nepotpunih redova.

```
try:
    kwargs = dict(
        dataset=dataset,
        metrics=[metric],
        raise_exceptions=True,
    )
    if run_cfg:
        kwargs["run_config"] = run_cfg

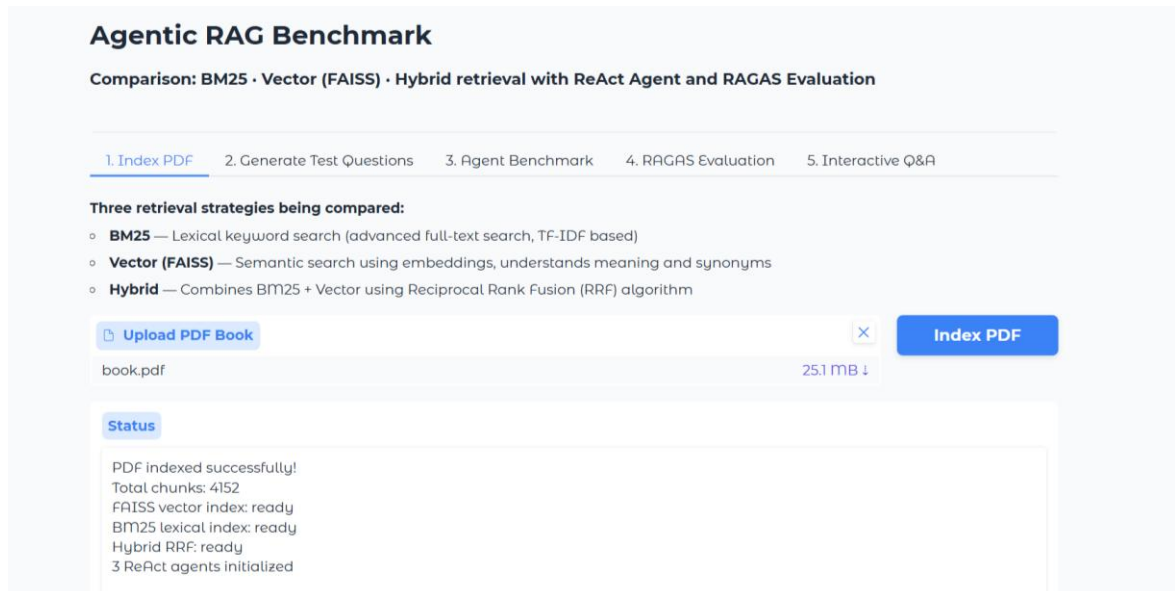
    result = evaluate(**kwargs)
    df = result.to_pandas()
```

Slika 14: Pokretanje RAGAS evaluacije (iz `evaluator.py`).

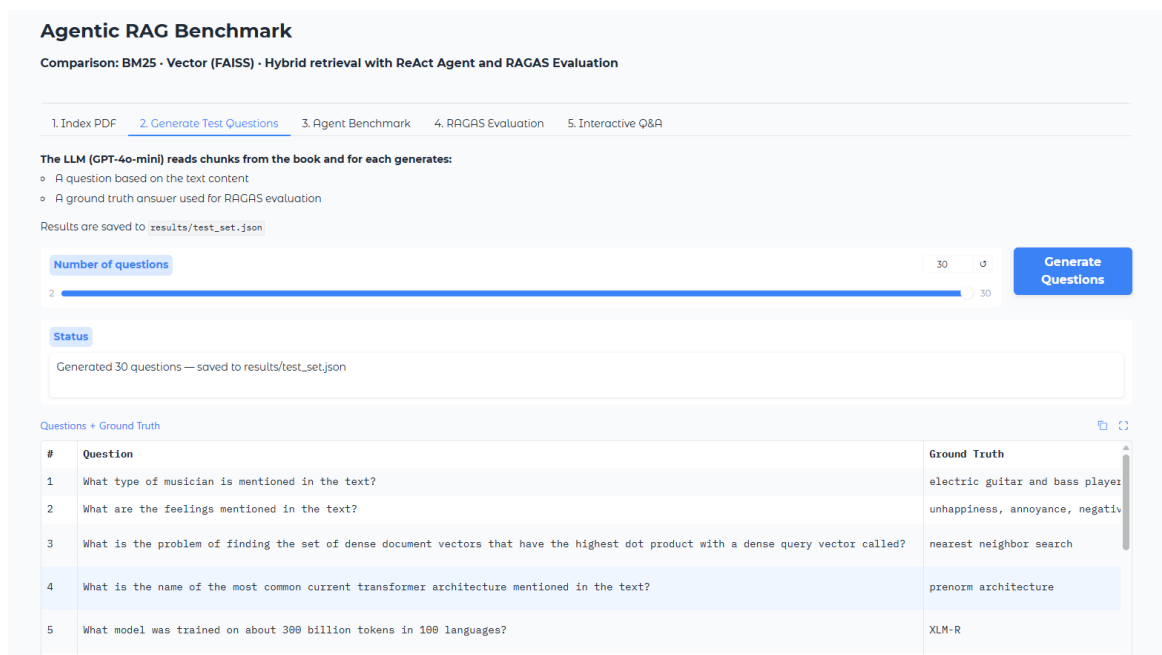
5.3.5 Korisnički interfejs

Interfejs je realizovan bibliotekom Gradio kroz pet kartica koje prate prirodan tok rada: (1) indeksiranje PDF dokumenta, (2) generisanje test pitanja, (3) benchmark sva tri agenta nad

istim pitanjima, (4) RAGAS evaluacija i (5) interaktivni razgovor sa izborom strategije. Aplikacija održava stanje (indeksi, agenti, test skup, rezultati) i prikazuje napredak svake dugotrajne operacije, a rezultate snima u direktorijum results u obliku CSV datoteka koje su poslužile kao osnov analize u poglavlju 6.



Slika 15: Izgled Gradio interfejsa: indeksiranje dokumenta



Slika 16: Izgled Gradio interfejsa - generisanje test pitanja

Agentic RAG Benchmark

Comparison: BM25 · Vector (FAISS) · Hybrid retrieval with ReAct Agent and RAGAS Evaluation

1. Index PDF
2. Generate Test Questions
3. Agent Benchmark
4. RAGAS Evaluation
5. Interactive Q&A

All 3 agents answer the same questions — compare answers against ground truth directly

Run Benchmark (all 3 agents)

Status

Benchmark complete: 30 questions x 3 agents = 90 answers
Saved to results/raw_*.json

BM25 Answers
Vector Answers
Hybrid Answers

BM25 Agent Results

#	Question	Ground Truth
1	What type of musician is mentioned in the text?	electric guitar and bass player
2	What are the feelings mentioned in the text?	unhappiness, annoyance, negativ
3	What is the problem of finding the set of dense document vectors that have the highest dot product with a dense query vector called?	nearest neighbor search
4	What is the name of the most common current transformer architecture mentioned in the text?	prenorm architecture
5	What model was trained on about 300 billion tokens in 100 languages?	XLM-R
6	What does $P(y = 1 x)$ represent in the context of sentiment analysis?	The probability that the docume

Slika 17: Izgled Gradio interfejsa - agentski benchmark

Agentic RAG Benchmark

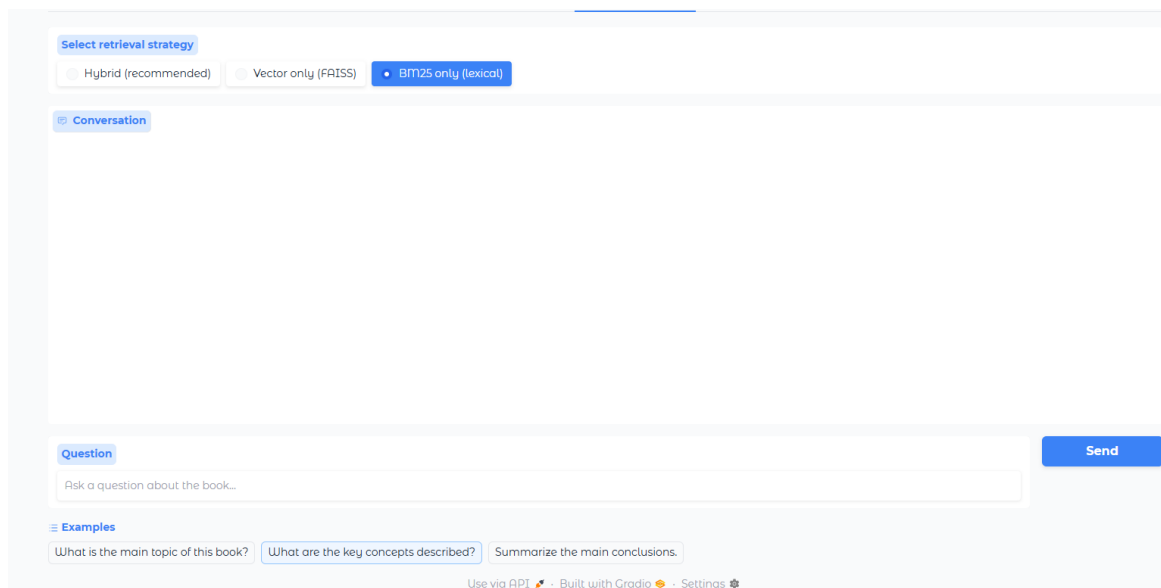
Comparison: BM25 · Vector (FAISS) · Hybrid retrieval with ReAct Agent and RAGAS Evaluation

1. Index PDF
2. Generate Test Questions
3. Agent Benchmark
4. RAGAS Evaluation
5. Interactive Q&A

Automatic evaluation of all 3 retrieval strategies:

Metric	Description
Faithfulness	Answer does not hallucinate beyond the retrieved context
Answer Relevancy	Answer actually addresses the question asked
Context Recall	All necessary documents were retrieved
Context Precision	Retrieved documents are relevant (low noise)

Slika 18: Izgled Gradio interfejsa - RAGAS evaluacija



Slika 19: Izgled Gradio interfejsa - interaktivno časkanje

5.4 Problemi tokom implementacije

Tokom razvoja rešeno je nekoliko netrivialnih problema, koji ilustruju realne izazove integracije savremenih biblioteka:

- Promene u LangChain-u između verzija 0.2 i 0.3 (premeštanje uvoza iz `langchain.schema` u `langchain_core`) rešene su pinovanjem verzija i odbrambenim uvozom (`try/except`) za delilac teksta.
- Zastarevanje argumenta `proxies` u `httpx` ≥ 0.28 rešeno je prelaskom na klasu `HTTPTransport` sa eksplicitnim proksijem.
- Konfiguracija RAGAS-a za Azure endpoint zahtevala je omotače `LangchainLLMWrapper` i `LangchainEmbeddingsWrapper` i izvršavanje svih metrika odjednom.
- Blokiranje adrese 0.0.0.0 od strane korporativnog proksija rešeno je pokretanjem Gradio servera na 127.0.0.1 uz postavljanje `NO_PROXY`.
- Promena rukovanja temom i CSS-om u Gradio 6.0 (prelazak iz konstruktora u poziv `launch()`) prilagođena je novoj specifikaciji.

5.5 Analiza modula za konfiguraciju

Modul `config.py` centralizuje sve parametre i uspostavlja vezu sa servisima za jezički model i ugradnje. Pored numeričkih parametara opisanih u poglavlju 4, ovaj modul rešava i niz tehničkih izazova specifičnih za korporativno okruženje u kome je sistem razvijan. Pristup modelu ostvaruje se preko Azure OpenAI servisa, što zahteva poseban oblik osnovnog URL-a (sa sufiksom `/api`), verziju API-ja i ime raspoređenog modela koje se razlikuje od javnog imena modela.

```
def get_llm(temperature: float = 0.0) -> AzureChatOpenAI:
    return AzureChatOpenAI(
        azure_deployment=CHAT_MODEL,
        azure_endpoint=ENDPOINT,
        api_key=API_KEY,
        api_version=API_VERSION,
        temperature=temperature,
        http_client=_make_http_client(),
        http_async_client=_make_async_http_client(),
    )
```

Slika 20: Fabrika jezičkog modela sa proksijem (iz `config.py`).

Funkcija `get_llm` prima temperaturu kao argument, čime se ista fabrika koristi i za deterministički rad (temperatura 0) i za generisanje pitanja (temperatura 0,3). Prosleđivanje prilagođenog HTTP klijenta sa proksijem (`_make_http_client`) ključno je za rad iza korporativnog mrežnog sloja; bez njega, pozivi servisu padaju sa greškom u vezi, što je, kako se vidi u rezultatima, povremeno pogađalo i samu vektorsku pretragu kada bi ugradnja upita naišla na prolaznu mrežnu grešku.

Funkcija `get_embeddings` analogno konstruiše klijenta za model ugradnji `text-embedding-3-small`. Pošto se i indeksiranje segmenata i ugradnja upita oslanjaju na isti servis, svaka nestabilnost tog servisa direktno utiče na vektorsku i (delimično) hibridnu komponentu, dok je leksička komponenta potpuno imuna jer ne zavisi ni od kakvog spoljašnjeg poziva. Ova asimetrija u zavisnosti od infrastrukture pokazaće se kao važan činilac u tumačenju rezultata.

5.6 Formatiranje konteksta i navođenje izvora

Važan deo implementacije jeste formatiranje pronađenih segmenata pre njihovog prosleđivanja modelu. Pomoćna funkcija `_fmt` spaja segmente i svakom dodaje oznaku izvora sa brojem strane, čime se obezbeđuje praćenje odgovora do konkretnog mesta u dokumentu. Time se

dodatno osnažuje osnovna prednost RAG pristupa, proverljivost, jer korisnik ili evaluator može da utvrdi tačno poreklo svake tvrdnje.

```
def _fmt(docs: List[Document]) -> str:
    parts = []
    for i, d in enumerate(docs, 1):
        src = d.metadata.get("source", "?")
        page = d.metadata.get("page", "?")
        parts.append(f"[Izvor {i} | str.{page} | {src}]\n{d.page_content}")
    return "\n\n---\n\n".join(parts)
```

Slika 21: Formatiranje konteksta sa oznakom izvora (iz `tool_factory.py`).

Kada nijedan segment nije pronađen, funkcija vraća eksplicitnu poruku umesto praznog niza. Takva implementacija utiče na rezultate evaluacije: agent koji primi ovakvu poruku može da odgovori da dokument ne sadrži traženu informaciju, umesto da generiše halucinaciju. Na pitanjima kod kojih je vraćena ova poruka razlike između strategija najjasnije se uočavaju, jer je vektorska strategija takvu poruku vraćala znatno češće od hibridne.

5.7 Komandno-linijski tok i ponovljivost

Pored Gradio interfejsa, sistem poseduje i komandno-linijsku ulaznu tačku (`main.py`), koja podržava neinteraktivno, skriptabilno pokretanje celokupnog cevovoda: indeksiranje, generisanje pitanja, benchmark i evaluaciju. Ovaj režim je presudan za ponovljivost eksperimenata, jer uklanja ljudski faktor iz pokretanja i obezbeđuje da se svaki eksperiment izvede pod identičnim uslovima. Rezultati se deterministički upisuju u direktorijum `results` u obliku CSV datoteka, čija struktura (po jedan red za svako pitanje, sa kolonama za pitanje, pronađeni kontekst, odgovor, referencu i četiri metrike) direktno odgovara podacima analiziranim u poglavlju 6.

Struktura izlaznih datoteka pažljivo je projektovana da bude samodovoljna: svaka datoteka sadrži ne samo numeričke metrike, već i pun tekst pronađenog konteksta i generisanog odgovora. Time je omogućena naknadna kvalitativna analiza, na primer, ručno čitanje zašto je neki odgovor dobio nisku vernost, bez potrebe za ponovnim pokretanjem skupih poziva modela. Ovako organizovana struktura izlaza omogućila je analizu po pojedinačnim pitanjima koja sledi u nastavku.

5.8 Detaljan prikaz izgradnje vektorskog indeksa

Izgradnja vektorskog indeksa odvija se u dva koraka: ugradnja svih segmenata i njihovo smeštanje u FAISS strukturu. Funkcija `build_vector_store` enkapsulira ovaj postupak, koristeći isti klijent za ugradnje koji je konfigurisan u `config.py` (sa proksijem i odgovarajućim vremenskim ograničenjem). Pošto se ugradnja svih segmenata izvodi jednom, u fazi pripreme, njen trošak se amortizuje preko svih kasnijih upita.

Poziv `FAISS.from_documents` za svaki segment poziva servis za ugradnju, prima vektor dimenzije 1536 i gradi indeks pogodan za pretragu najbližih suseda. Ovaj korak, masovna ugradnja segmenata, najosetljiviji na prolazne mrežne greške opisane ranije: ako poziv servisu povremeno otkáže, deo segmenata može ostati bez ispravne ugradnje, što degradira kvalitet vektorske pretrage. Leksički indeks, koji se gradi potpuno lokalno, imun je na ovu klasu grešaka.

Pretraga nad izgrađenim skladištem svodi se na ugradnju upita i pronalaženje k najbližih vektora po kosinusnoj sličnosti. Ova simetrija, isti model ugrađuje i segmente i upit, ključna je za ispravnost: pretraga je smisljena samo ako su segmenti i upit u istom vektorskom prostoru, što je obezbeđeno upotrebom istog modela za ugradnju na oba kraja.

5.9 Priprema skupa podataka za evaluaciju

Pre poziva RAGAS-u, rezultati agenata moraju se preoblikovati u strukturu koju okvir očekuje, skup podataka sa kolonama za pitanje, generisani odgovor, listu pronađenih konteksta i tačan odgovor (referencu). Funkcija `_build_dataset` obavlja ovo preoblikovanje uz odbrambeno filtriranje koje osigurava da svaki red sadrži sve potrebne kolone, čime se sprečavaju rušenja zbog nepotpunih unosa.

```

def _build_dataset(test_results: List[Dict[str, Any]]) -> Dataset:
    """
    New RAGAS versions expect:
    user_input      (was: question)
    response        (was: answer)
    retrieved_contexts (was: contexts)
    reference       (was: ground_truth)
    """
    rows = {
        "user_input": [],
        "response": [],
        "retrieved_contexts": [],
        "reference": [],
    }
    for r in test_results:
        rows["user_input"].append(r.get("question", ""))
        rows["response"].append(r.get("answer", ""))
        ctx = r.get("contexts", [])
        if isinstance(ctx, str):
            ctx = [ctx]
        ctx = [str(c) for c in ctx if c]
        rows["retrieved_contexts"].append(ctx if ctx else ["no context retrieved"])
        rows["reference"].append(r.get("ground_truth", ""))
    return Dataset.from_dict(rows)

```

Slika 22: Priprema skupa podataka za RAGAS (iz evaluator.py).

Odluka da se nepotpuni redovi preskoče, a ne da se popune zamenskim vrednostima, doneta je svesno: ubacivanje veštačkih vrednosti iskrivilo bi metrike, dok preskakanje čuva integritet merenja po cenu nešto manjeg efektivnog uzorka za pojedine strategije. U praksi, pošto sistem za svaki promašaj beleži eksplicitnu poruku o nepronadjenom kontekstu (a ne prazan niz), većina redova ipak prolazi filter sa kontekstom koji jasno signalizira neuspeh pretrage, što omogućava da se i takvi slučajevi korektno ocene niskim vrednostima umesto da budu potpuno izostavljeni.

6. EKSPERIMENTALNA EVALUACIJA

U ovom poglavlju izlažu se eksperimentalna postavka, izmereni rezultati po metrikama i pitanjima, njihova analiza i provera postavljenih hipoteza. Svi rezultati potiču iz stvarnog izvršavanja sistema i sačuvani su u CSV datotekama (ragas_bm25.csv, ragas_vector.csv, ragas_hybrid.csv).

6.1 Eksperimentalna postavka

6.1.1 Dokument i korpus

Kao testni dokument korišćena je knjiga Speech and Language Processing autora Daniela Jurafskog i Džejmisa Martina, obima 626 strana. Reč je o jednom od najuticajnijih udžbenika iz oblasti obrade prirodnog jezika i računarske lingvistike, koji pokriva teme od osnovne lingvistike do velikih jezičkih modela. Domen je tehnički i terminološki bogat, što ga čini zahtevnim i reprezentativnim test-korpusom za RAG sistem.

Nakon podele na segmente veličine 512 tokena uz preklapanje od 64 tokena, dokument je raščlanjen na nekoliko hiljada segmenata (red veličine 4×10^3 , procenjeno na osnovu ukupnog broja karaktera i koraka delioca), koji su paralelno indeksirani u FAISS i BM25 indeks.

6.1.2 Test skup

Test skup čini 30 sintetički generisanih pitanja, svako sa pripadajućim tačnim odgovorom (ground truth) izvučenim iz nasumično odabranog segmenta. Pitanja pokrivaju širok raspon tipova: faktografska pitanja (npr. „Koliko transkribovanih razgovora sadrži HKUST korpus?“), pojmovna pitanja (npr. „Koji je cilj chain-of-thought tehnike?“), pitanja o autorstvu i bibliografiji, kao i pitanja koja zahtevaju razumevanje matematičkih formula (npr. formula za glas anchora u entity linking-u). Tabela 6 navodi pet reprezentativnih primera.

Tabela 6: Reprezentativni primeri pitanja iz test skupa.

Pitanje (engl. original)	Tip
What values are set to $-\infty$ in the $N \times N$ QKT matrix?	Faktografsko / matematičko
What is the goal of chain-of-thought prompting?	Pojmovno
Who are the authors of 'Anaphora for everyone'?	Bibliografsko
What type of audio does HuBERT use as input?	Faktografsko
What is the formula for the vote given by anchor b...?	Matematičko

6.1.3 Parametri evaluacije

Radi ponovljivosti, svi eksperimenti izvedeni su pod istim uslovima: temperatura modela 0,0 (deterministički odgovori), top-K = 5 za sve strategije, pondera fuzije $\alpha = 0,5$ (jednaka težina vektorske i leksičke liste), maksimalno 6 iteracija agenta i ograničenje izvršavanja od 120 sekundi po pitanju. Isti model (GPT-4o-mini) korišćen je i za generisanje odgovora i kao sudija u RAGAS evaluaciji.

Radi pune ponovljivosti, ovde se navodi tačan redosled koraka kojim su rezultati dobijeni. Protokol je deterministički i može se ponoviti pokretanjem komandno-linijskog toka nad istim korpusom i istim test skupom.

1. Učitavanje korpusa: PDF dokument (book.pdf) učitava se PyMuPDF učitavačem uz očuvanje brojeva strana.
2. Podela na segmente: rekurzivni delilac deli tekst na segmente od 512 tokena uz preklapanje od 64 tokena.
3. Indeksiranje: isti skup segmenata indeksira se paralelno u FAISS (gusti vektori) i BM25 (invertovani indeks).
4. Generisanje test skupa: iz nasumično uzorkovanih segmenata generiše se 30 parova pitanje, tačan odgovor (temperatura 0,3).
5. Izvršavanje agenata: za svako pitanje izvršavaju se sva tri agenta (leksički, vektorski, hibridni) pod identičnim uslovima (temperatura 0,0; top-K = 5; $\alpha = 0,5$).
6. Beleženje konteksta: pronađeni kontekst i odgovor svakog agenta beleže se zajedno sa međukoracima.
7. RAGAS evaluacija: za svaki par (odgovor, kontekst, referenca) izračunavaju se sve četiri metrike odjednom (model-sudija na temperaturi 0,0).
8. Snimanje: rezultati se upisuju u CSV datoteke koje su poslužile kao osnov analize u ovom poglavlju.

Ključna odlika protokola je da je jedina promenljiva koja se menja između tri tretmana jeste strategija pretrage; sve ostalo (korpus, segmenti, pitanja, model, temperatura, top-K) ostaje nepromenjeno. Time je obezbeđena unutrašnja valjanost poređenja u meri u kojoj to dozvoljavaju zabeležene infrastrukturne greške, o kojima se diskutuje u poglavlju 7.

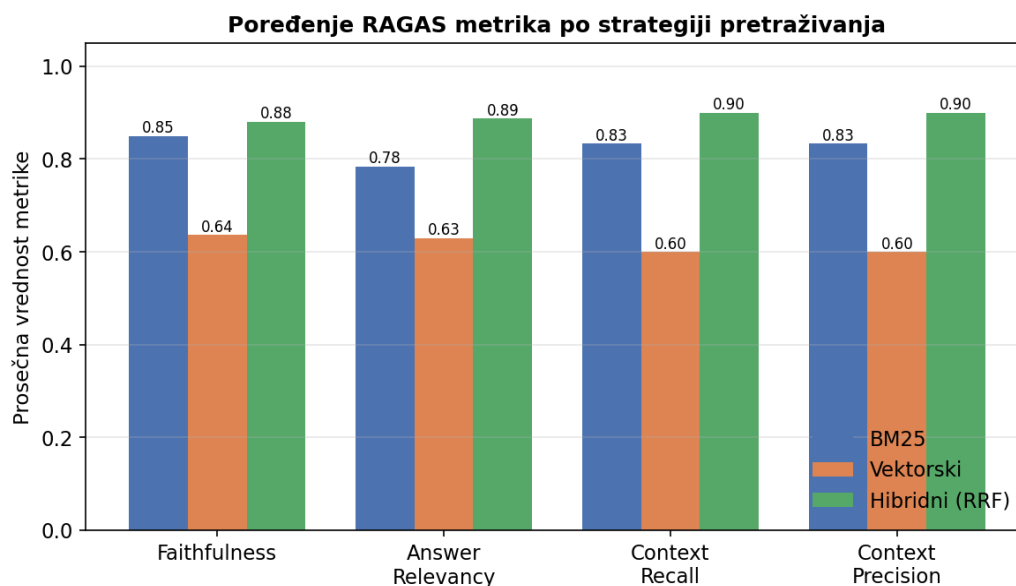
Test skup od 30 pitanja predstavlja relativno mali uzorak, pa dobijene rezultate treba tumačiti oprezno i uzeti sa zadrškom. Oni daju indiciju da je jedan pristup bolji od drugog, ali za pouzdano prihvatanje statističke značajnosti bilo bi potrebno sprovesti detaljnije testiranje, sa većim brojem pitanja i više ponavljanja.

6.2 Rezultati po metrikama

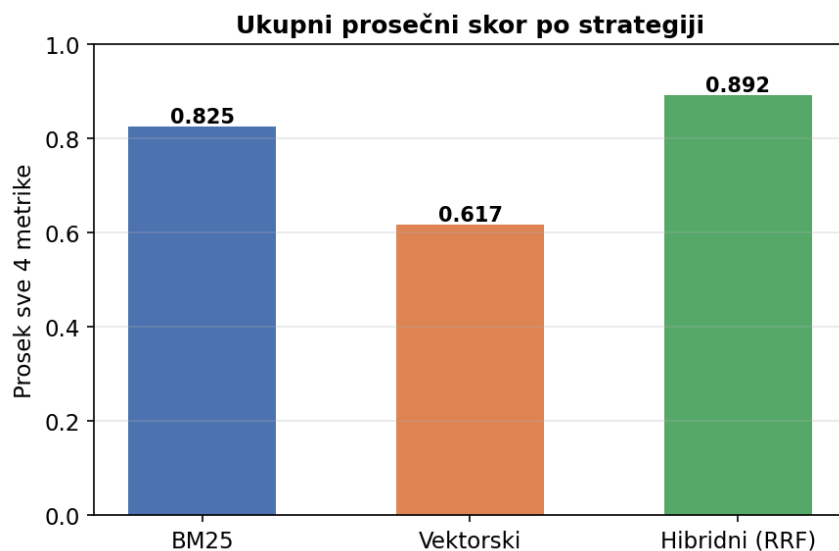
Tabela 7 prikazuje zbirne rezultate, prosečne vrednosti sve četiri metrike za tri strategije. Hibridna strategija postiže najviše vrednosti na svim metrikama.

Tabela 7: Zbirne prosečne vrednosti RAGAS metrika po strategiji.

Metrika	BM25	Vektorski	Hibridni	Pobednik
Faithfulness	0,850	0,637	0,881	Hibridni
Answer Relevancy	0,784	0,630	0,887	Hibridni
Context Recall	0,833	0,600	0,900	Hibridni
Context Precision	0,833	0,600	0,900	Hibridni
Prosek (sve 4)	0,825	0,617	0,892	Hibridni



Slika 23: Grupisani prikaz prosečnih vrednosti metrika po strategiji.



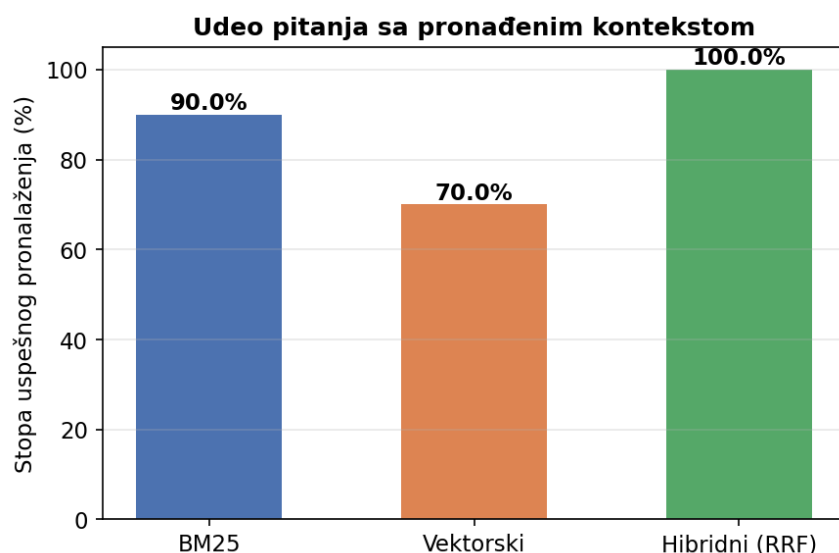
Slika 24: Ukupni prosečni skor (prosek sve četiri metrike) po strategiji.

Tabela 8 prikazuje standardnu devijaciju (populaciona) po metrici, koja otkriva koliko su rezultati stabilni. Hibridna strategija ima ne samo najviši prosek, već i najmanju varijabilnost na svim metrikama, što ukazuje na doslednost.

Tabela 8: Standardna devijacija metrika po strategiji (niže je bolje).

Metrika (st. dev.)	BM25	Vektorski	Hibridni
Faithfulness	0,345	0,464	0,274
Answer Relevancy	0,325	0,449	0,232
Context Recall	0,373	0,490	0,300
Context Precision	0,373	0,490	0,300

Posebno indikativan pokazatelj je stopa uspešnog pronalaženja konteksta, udeo pitanja za koja je strategija uopšte vratila kontekst. Vektorska strategija nije pronašla kontekst za 9 od 30 pitanja (30%), BM25 za 3 od 30 (10%), dok hibridna strategija nije imala nijedan takav slučaj (0%). Slika 6.3 to prikazuje.



Slika 25: Udeo pitanja za koja je pronađen kontekst, po strategiji.

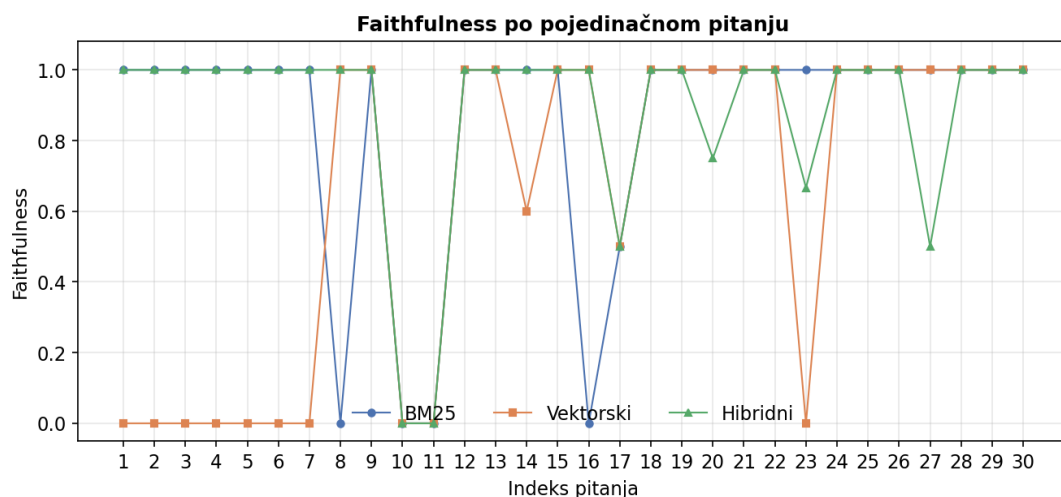
6.2.1 Faithfulness, analiza po pitanju

Tabela 9 prikazuje vrednost metrike faithfulness za svako od 30 pitanja i sve tri strategije. Vidljivo je da hibridna strategija retko pada na nulu, dok vektorska ima više potpunih promašaja.

Tabela 9: Faithfulness po pojedinačnom pitanju.

#	Pitanje (skraćeno)	BM25	Vektor	Hibrid
1	What values are set to $-\infty$ in the $N \times N$ QKT matrix?	1.00	0.00	1.00
2	What type of optimization problem is learning in ne...	1.00	0.00	1.00
3	What is the second step in the process described in...	1.00	0.00	1.00
4	How many instruction/response instances are include...	1.00	0.00	1.00
5	What is the selectional restriction on the THEME ro...	1.00	0.00	1.00
6	What type of parsers will be discussed in relation ...	1.00	0.00	1.00
7	How many transcribed conversations are included in ...	1.00	0.00	1.00
8	Who are the authors of the paper on knowledge-based...	0.00	1.00	1.00
9	How many instructions does the Aya dataset provide?	1.00	1.00	1.00
10	What is the distance of the satellite's orbit from ...	0.00	0.00	0.00
11	What type of musician is mentioned in the text?	0.00	0.00	0.00
12	What is the title of the paper by Prates, Avelar, a...	1.00	1.00	1.00
13	Who are the authors of the paper titled 'Anaphora f...	1.00	1.00	1.00
14	What are the main dental sounds in English?	1.00	0.60	1.00
15	What does the decoder generate from the quantizer o...	1.00	1.00	1.00
16	What is the role of the phrase 'someplace nearby' i...	0.00	1.00	1.00

#	Pitanje (skraćeno)	BM25	Vektor	Hibrid
17	What is the task called that involves detecting whe...	0.50	0.50	0.50
18	What does the activation value of the hidden layer ...	1.00	1.00	1.00
19	How many times is a cell selected from row x to cre...	1.00	1.00	1.00
20	What is the context window size for transformer-bas...	1.00	1.00	0.75
21	What is the training signal for the model in langua...	1.00	1.00	1.00
22	What is the goal of chain-of-thought prompting?	1.00	1.00	1.00
23	What is the simplest and most influential architect...	1.00	0.00	0.67
24	What was the influential entity-based system mentio...	1.00	1.00	1.00
25	What are some sources from which the process might ...	1.00	1.00	1.00
26	Who showed the equivalence of the WFST and the Earl...	1.00	1.00	1.00
27	What is the purpose of calculating the standard dev...	1.00	1.00	0.50
28	What is the formula for the vote given by anchor b ...	1.00	1.00	1.00
29	What type of audio does HuBERT use as input?	1.00	1.00	1.00
30	Who introduced recurrence at the hidden layer in a ...	1.00	1.00	1.00
	PROSEK	0.850	0.637	0.881



Slika 26: Faithfulness po pojedinačnom pitanju (linijski prikaz).

6.2.2 Answer Relevancy, analiza po pitanju

Tabela 10 prikazuje relevantnost odgovora po pitanju. Hibridna strategija dominira, posebno kod pitanja gde druge strategije nisu pronašle kontekst (relevantnost tada pada na nulu).

Tabela 10: Answer relevancy po pojedinačnom pitanju.

#	Pitanje (skraćeno)	BM25	Vektor	Hibrid
1	What values are set to $-\infty$ in the $N \times N$ QKT matrix?	0.64	0.00	0.91
2	What type of optimization problem is learning in ne...	0.70	0.00	0.70

#	Pitanje (skraćeno)	BM25	Vektor	Hibrid
3	What is the second step in the process described in...	0.52	0.00	0.90
4	How many instruction/response instances are include...	1.00	0.00	1.00
5	What is the selectional restriction on the THEME ro...	0.91	0.00	0.89
6	What type of parsers will be discussed in relation ...	0.70	0.00	0.97
7	How many transcribed conversations are included in ...	0.98	0.00	0.98
8	Who are the authors of the paper on knowledge-based...	0.00	1.00	1.00
9	How many instructions does the Aya dataset provide?	0.74	0.74	0.74
10	What is the distance of the satellite's orbit from ...	0.95	0.05	0.95
11	What type of musician is mentioned in the text?	0.00	0.00	0.17
12	What is the title of the paper by Prates, Avelar, a...	0.99	1.00	0.99
13	Who are the authors of the paper titled 'Anaphora f...	1.00	1.00	1.00
14	What are the main dental sounds in English?	1.00	1.00	1.00
15	What does the decoder generate from the quantizer o...	1.00	1.00	1.00
16	What is the role of the phrase 'someplace nearby' i...	0.11	0.72	0.89
17	What is the task called that involves detecting whe...	1.00	1.00	1.00
18	What does the activation value of the hidden layer ...	1.00	1.00	1.00
19	How many times is a cell selected from row x to cre...	0.74	0.74	0.74
20	What is the context window size for transformer-bas...	1.00	1.00	1.00
21	What is the training signal for the model in langua...	0.94	0.91	1.00
22	What is the goal of chain-of-thought prompting?	1.00	1.00	1.00
23	What is the simplest and most influential architect...	1.00	1.00	1.00
24	What was the influential entity-based system mentio...	0.96	0.95	0.96
25	What are some sources from which the process might ...	0.00	0.00	0.00
26	Who showed the equivalence of the WFST and the Earl...	0.79	0.79	0.82
27	What is the purpose of calculating the standard dev...	1.00	1.00	1.00
28	What is the formula for the vote given by anchor b ...	1.00	1.00	1.00
29	What type of audio does HuBERT use as input?	0.84	1.00	1.00
30	Who introduced recurrence at the hidden layer in a ...	1.00	1.00	1.00
	PROSEK	0.784	0.630	0.887

6.2.3 Context Recall i Context Precision, analiza po pitanju

Tabele 11 i 12 prikazuju odziv i preciznost konteksta. Na ovom test skupu, ove dve metrike imaju identične vrednosti po pitanju (i isti prosek), što je posledica binarne prirode većine pitanja: kada se pronađe pravi segment, oba pokazatelja iznose 1; kada se ne pronađe, oba iznose 0.

Tabela 11: Context recall po pojedinačnom pitanju.

#	Pitanje (skraćeno)	BM25	Vektor	Hibrid
1	What values are set to $-\infty$ in the $N \times N$ QKT matrix?	1.00	0.00	1.00
2	What type of optimization problem is learning in ne...	1.00	0.00	1.00
3	What is the second step in the process described in...	0.00	0.00	0.00
4	How many instruction/response instances are include...	1.00	0.00	1.00
5	What is the selectional restriction on the THEME ro...	1.00	0.00	1.00
6	What type of parsers will be discussed in relation ...	0.00	0.00	1.00
7	How many transcribed conversations are included in ...	1.00	0.00	1.00
8	Who are the authors of the paper on knowledge-based...	0.00	1.00	1.00
9	How many instructions does the Aya dataset provide?	1.00	1.00	1.00
10	What is the distance of the satellite's orbit from ...	1.00	0.00	1.00
11	What type of musician is mentioned in the text?	0.00	0.00	1.00
12	What is the title of the paper by Prates, Avelar, a...	1.00	1.00	1.00
13	Who are the authors of the paper titled 'Anaphora f...	1.00	1.00	1.00
14	What are the main dental sounds in English?	1.00	1.00	1.00
15	What does the decoder generate from the quantizer o...	1.00	1.00	1.00
16	What is the role of the phrase 'someplace nearby' i...	0.00	1.00	1.00
17	What is the task called that involves detecting whe...	1.00	1.00	1.00
18	What does the activation value of the hidden layer ...	1.00	1.00	1.00
19	How many times is a cell selected from row x to cre...	1.00	1.00	1.00
20	What is the context window size for transformer-bas...	1.00	1.00	1.00
21	What is the training signal for the model in langua...	1.00	0.00	0.00
22	What is the goal of chain-of-thought prompting?	1.00	1.00	1.00
23	What is the simplest and most influential architect...	1.00	0.00	0.00
24	What was the influential entity-based system mentio...	1.00	0.00	1.00
25	What are some sources from which the process might ...	1.00	1.00	1.00
26	Who showed the equivalence of the WFST and the Earl...	1.00	1.00	1.00
27	What is the purpose of calculating the standard dev...	1.00	1.00	1.00
28	What is the formula for the vote given by anchor b ...	1.00	1.00	1.00
29	What type of audio does HuBERT use as input?	1.00	1.00	1.00
30	Who introduced recurrence at the hidden layer in a ...	1.00	1.00	1.00
	PROSEK	0.833	0.600	0.900

Tabela 12: Context precision po pojedinačnom pitanju.

#	Pitanje (skraćeno)	BM25	Vektor	Hibrid
1	What values are set to $-\infty$ in the $N \times N$ QKT matrix?	1.00	0.00	1.00
2	What type of optimization problem is learning in ne...	1.00	0.00	1.00
3	What is the second step in the process described in...	0.00	0.00	0.00
4	How many instruction/response instances are include...	1.00	0.00	1.00
5	What is the selectional restriction on the THEME ro...	1.00	0.00	1.00
6	What type of parsers will be discussed in relation ...	0.00	0.00	1.00
7	How many transcribed conversations are included in ...	1.00	0.00	1.00
8	Who are the authors of the paper on knowledge-based...	0.00	1.00	1.00
9	How many instructions does the Aya dataset provide?	1.00	1.00	1.00
10	What is the distance of the satellite's orbit from ...	1.00	0.00	1.00
11	What type of musician is mentioned in the text?	0.00	0.00	1.00
12	What is the title of the paper by Prates, Avelar, a...	1.00	1.00	1.00
13	Who are the authors of the paper titled 'Anaphora f...	1.00	1.00	1.00
14	What are the main dental sounds in English?	1.00	1.00	1.00
15	What does the decoder generate from the quantizer o...	1.00	1.00	1.00
16	What is the role of the phrase 'someplace nearby' i...	0.00	1.00	1.00
17	What is the task called that involves detecting whe...	1.00	1.00	1.00
18	What does the activation value of the hidden layer ...	1.00	1.00	1.00
19	How many times is a cell selected from row x to cre...	1.00	1.00	1.00
20	What is the context window size for transformer-bas...	1.00	1.00	1.00
21	What is the training signal for the model in langua...	1.00	0.00	0.00
22	What is the goal of chain-of-thought prompting?	1.00	1.00	1.00
23	What is the simplest and most influential architect...	1.00	0.00	0.00
24	What was the influential entity-based system mentio...	1.00	0.00	1.00
25	What are some sources from which the process might ...	1.00	1.00	1.00
26	Who showed the equivalence of the WFST and the Earl...	1.00	1.00	1.00
27	What is the purpose of calculating the standard dev...	1.00	1.00	1.00
28	What is the formula for the vote given by anchor b ...	1.00	1.00	1.00
29	What type of audio does HuBERT use as input?	1.00	1.00	1.00
30	Who introduced recurrence at the hidden layer in a ...	1.00	1.00	1.00
	PROSEK	0.833	0.600	0.900

6.3 Analiza rezultata

6.3.1 Kada pobeđuje BM25

BM25 se pokazao iznenađujuće snažnim, što je u skladu sa upozorenjem iznetim u radu [14] o potcenjivanju leksičkih osnovica. Leksička pretraga dominira kod pitanja sa egzaktnim terminima, imenima, brojevima i datumima. Karakterističan primer je pitanje „Koliko transkribovanih razgovora sadrži HKUST Mandarin korpus?“, gde se tačan odgovor (1206) nalazi u segmentu koji doslovno sadrži ključne termine pitanja; BM25 ga pronalazi sa savršenim skorom, dok vektorska pretraga u datoj konfiguraciji za to pitanje uopšte nije vratila kontekst. Slično važi za pitanja o tačnim brojevima (npr. 204K instanci u Aya korpusu) i autorstvu radova.

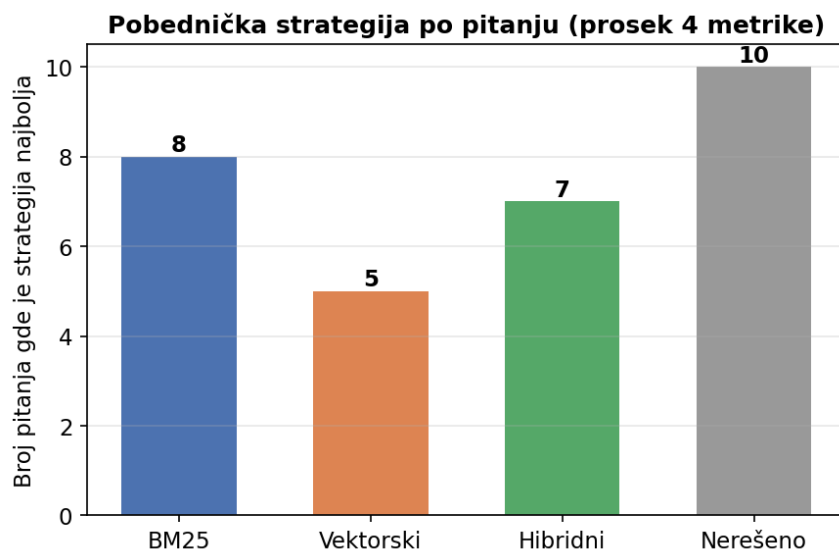
6.3.2 Kada pobeđuje vektorska pretraga

Vektorska pretraga, kada uspe da pronađe kontekst, postiže visoke vrednosti relevantnosti na pojmovnim pitanjima i parafrazama. Na primer, kod pitanja o ulozi fraze „someplace nearby“, vektorska strategija pronalazi semantički relevantan segment i daje potpuno relevantan odgovor. Međutim, njena glavna slabost u ovom eksperimentu nije u kvalitetu kada uspe, već u učestalosti potpunih promašaja: kod 9 pitanja nije vratila nikakav kontekst, što je oborilo njen prosek. Ovo upozorava da samostalna vektorska pretraga, bez rezervne leksičke komponente, može biti krhka.

Važno je naglasiti da deo neuspeha vektorske strategije potiče i od infrastrukturnih grešaka (zabeležene poruke o grešci u vezi prilikom poziva servisa za ugradnju), a ne isključivo od inherentne slabosti metode. Ovo ograničenje eksplicitno se razmatra u poglavlju 7.

6.3.3 Prednost hibridne pretrage

Hibridna strategija pokazuje jasnu prednost u robusnosti. Kombinujući dve liste, ona praktično eliminiše slučajeve potpunog izostanka konteksta: ako jedna strategija promaši, druga gotovo uvek nadoknadi. Posledica je najviši prosek na svim metrikama i najmanja varijabilnost. Analiza po pitanju (Slika 27) pokazuje da je hibridna strategija među najboljima (sa skorom većim ili jednakim ostalima) kod približno polovine pitanja, dok je broj pitanja gde je samostalno najbolja uravnotežen sa BM25, što je očekivano, jer hibrid nasleđuje snagu BM25 komponente i dodaje semantičku robusnost.



Slika 27: Broj pitanja na kojima je svaka strategija jedini pobednik (prosek 4 metrike; „nerešeno“ obuhvata pitanja sa izjednačenim maksimumom).

Analiza osetljivosti na pondera α nije bila predmet primarnog eksperimenta (korišćeno je $\alpha = 0,5$), ali predstavlja prirodan pravac proširenja: niže α naginje ka leksičkoj, više ka semantičkoj komponenti. S obzirom na snagu BM25 na ovom korpusu, moguće je da bi α nešto niže od 0,5 dodatno podiglo rezultate hibrida.

6.4 Provera hipoteza

H1 (potvrđena): Hibridni pretraživač postiže višu vrednost faithfulness (0,881) od BM25 (0,850) i vektorske pretrage (0,637) posmatranih pojedinačno. Razlika u odnosu na vektorsku je velika, a u odnosu na BM25 umerena ali dosledna.

H2 (delimično potvrđena / nijansirana): Vektorska pretraga jeste pokazala prednost na pojedinim semantički složenim pitanjima kada je uspela da pronađe kontekst, ali zbog visoke stope promašaja njen ukupni prosek je niži od BM25. Hipoteza, dakle, važi na nivou pojedinačnih pojmovnih pitanja, ali ne i na nivou ukupnog proseka u datoj konfiguraciji.

H3 (potvrđena): Hibridni agent, koji raspolaže sa više alata, postiže najvišu context precision (0,900), višu od jednoalatnih agenata (BM25 0,833; vektorski 0,600). Time se potvrđuje da mogućnost izbora i fuzije strategija doprinosi kvalitetu pronađenog konteksta.

6.5 Studije slučaja pojedinačnih pitanja

Agregatni proseci i standardne devijacije daju opštu sliku, ali se prava priroda razlika među strategijama najjasnije vidi na nivou pojedinačnih pitanja. U ovom odeljku detaljno se analizira šest reprezentativnih pitanja koja ilustruju karakteristične obrasce ponašanja: čistu pobedu leksičke pretrage, oporavak vektorske pretrage na parafrazi, zajednički neuspeh, vrednost fuzije, slučaj delimične vernosti i graničan slučaj. Za svako pitanje navode se izmerene vrednosti sve četiri metrike po strategiji.

6.5.1 Slučaj egzaktnog entiteta: HKUST Mandarin korpus

Tabela 13: Metrike za pitanje o broju razgovora u HKUST korpusu.

Pitanje 7	<i>How many transcribed conversations are included in the HKUST Mandarin Telephone Speech corpus?</i>
BM25	f=1.00 ar=0.98 cr=1.00 cp=1.00
Vektorski	f=0.00 ar=0.00 cr=0.00 cp=0.00
Hibridni	f=1.00 ar=0.98 cr=1.00 cp=1.00

Ovo pitanje je arhetip slučaja u kome leksička pretraga briljira. Pitanje sadrži veoma redak, specifičan entitet („HKUST Mandarin Telephone Speech corpus“) i traži konkretan broj (1206). BM25, vođen visokom inverznom učestalošću tih termina, gotovo nepogrešivo locira tačan segment i postiže savršene vrednosti svih metrika osim blage razlike u relevantnosti odgovora. Vektorska pretraga, nasuprot tome, za ovo pitanje uopšte nije vratila kontekst (sve metrike na nuli), gust vektor pitanja nije bio dovoljno blizu vektoru ciljnog segmenta, verovatno zato što je segment dominantno numerički i nazivni, a takav sadržaj se slabo razlikuje u semantičkom prostoru. Hibridna strategija nasleđuje uspeh BM25 komponente i takođe postiže savršen rezultat.

6.5.2 Slučaj parafraze: uloga fraze „someplace nearby“

Tabela 14: Metrike za pojmovno pitanje o ulozi fraze „someplace nearby“.

Pitanje 16	<i>What is the role of the phrase 'someplace nearby' in the sensible interpretation of the sentence?</i>
BM25	f=0.00 ar=0.11 cr=0.00 cp=0.00
Vektorski	f=1.00 ar=0.72 cr=1.00 cp=1.00
Hibridni	f=1.00 ar=0.89 cr=1.00 cp=1.00

Ovo pitanje je ogledni primer u prilog hipoteze H2. Reč je o pojmovnom, semantičkom pitanju o ulozi jedne fraze u tumačenju rečenice, bez retkih, doslovnih termina koji bi usmerili leksičku pretragu. Ovde BM25 potpuno promašuje (sve metrike na nuli), jer ne postoji jak leksički signal. Vektorska pretraga, naprotiv, uspeva: njen gust vektor hvata značenjsku srodnost i pronalazi pravi segment, postižući punu vernost i visoku relevantnost. Hibridna strategija ponovo objedinjuje najbolje od obe, postiže pun rezultat, jer je dovoljno da jedna komponenta (ovde vektorska) uspe. Ovaj slučaj pokazuje da prednost vektorske pretrage na semantičkim pitanjima postoji na nivou pojedinačnog pitanja, iako se, zbog čestih promašaja na drugim pitanjima, ne odražava u njenom ukupnom proseku.

6.5.3 Slučaj vrednosti fuzije: matrica QK^T

Tabela 15: Metrike za pitanje o vrednostima $-\infty$ u matrici QK^T .

Pitanje 1	<i>What values are set to $-\infty$ in the $N \times N$ QK^T matrix?</i>
BM25	f=1.00 ar=0.64 cr=1.00 cp=1.00
Vektorski	f=0.00 ar=0.00 cr=0.00 cp=0.00
Hibridni	f=1.00 ar=0.91 cr=1.00 cp=1.00

Pitanje o tome koje se vrednosti u matrici QK^T postavljaju na $-\infty$ (odgovor: vrednosti iznad dijagonale, radi kauzalnog maskiranja) kombinuje tehnički termin sa pojmovnim razumevanjem. Ovde BM25 uspeva zahvaljujući prisustvu specifične notacije, dok vektorska pretraga potpuno promašuje. Hibridna strategija ne samo da uspeva, već postiže i najvišu relevantnost odgovora (0,91) među svim strategijama na ovom pitanju, što sugerise da je, oslanjajući se na BM25 komponentu za pronalaženje uz dodatni semantički kontekst, formirala nešto bogatiji i precizniji odgovor. Ovo je primer u kome hibrid ne samo da izbegava promašaj, već donosi i marginalno poboljšanje kvaliteta odgovora.

6.5.4 Slučaj zajedničkog izazova: „najjednostavnija i najuticajnija arhitektura“

Tabela 16: Metrike za pitanje o najuticajnijoj arhitekturi.

Pitanje 23	<i>What is the simplest and most influential architecture discussed in the text?</i>
BM25	f=1.00 ar=1.00 cr=1.00 cp=1.00
Vektorski	f=0.00 ar=1.00 cr=0.00 cp=0.00
Hibridni	f=0.67 ar=1.00 cr=0.00 cp=0.00

Ovo pitanje ilustruje granicu mogućnosti fuzije. Formulacija je nejasna i kontekstualno zavisna („najjednostavnija i najuticajnija arhitektura diskutovana u tekstu“), bez jakog leksičkog ili jednoznačnog semantičkog sidrišta. Sve tri strategije ovde imaju poteškoće: BM25 postiže punu vernost ali nizak odziv konteksta, vektorska ima visok odgovor ali nultu vernost na nivou konteksta, a hibridna postiže delimičnu vernost (0,67) uz visoku relevantnost. Pouka je da fuzija ne može da stvori signal tamo gde ga nijedna komponenta nema u dovoljnoj meri; ona pomaže kada bar jedna strategija pronađe pravi segment, ali ne čini čuda kod inherentno dvosmislenih pitanja. Ovo je iskreno dokumentovano ograničenje, a ne propust koji treba prikriti.

6.5.5 Slučaj delimične vernosti: detekcija kraja govora

Tabela 17: Metrike za pitanje o detekciji završetka govora (endpointing).

Pitanje 17	<i>What is the task called that involves detecting whether a user is done speaking?</i>
BM25	f=0.50 ar=1.00 cr=1.00 cp=1.00
Vektorski	f=0.50 ar=1.00 cr=1.00 cp=1.00
Hibridni	f=0.50 ar=1.00 cr=1.00 cp=1.00

Kod ovog pitanja sve tri strategije pronalaze kontekst i postižu punu relevantnost i savršen odziv, ali vernost iznosi 0,5 kod svih. Ujednačenost ishoda ukazuje da je problem u fazi generisanja, a ne pronalaženja: pronađeni segment jeste relevantan, ali odgovor sadrži jednu tvrdnju koja nije u potpunosti poduprta kontekstom (otuda vernost tačno na pola). Ovaj slučaj je značajan jer pokazuje da, kada je kvalitet pronalaženja izjednačen, razlike među strategijama iščezavaju, potvrđujući ranije izvedenu tezu da je pronalaženje (faktor P_r) glavni izvor razlika, dok je generisanje (faktor P_g) zajedničko svim strategijama jer koriste isti model.

6.5.6 Granični slučaj: pitanje o „muzičaru“

Tabela 18: Metrike za nejasno pitanje o „muzičaru“ pomenutom u tekstu.

Pitanje 11	<i>What type of musician is mentioned in the text?</i>
BM25	f=0.00 ar=0.00 cr=0.00 cp=0.00
Vektorski	f=0.00 ar=0.00 cr=0.00 cp=0.00
Hibridni	f=0.00 ar=0.17 cr=1.00 cp=1.00

Pitanje „Koja vrsta muzičara se pominje u tekstu?“ nedovoljno je određeno i izvan šireg konteksta teško razumljivo, što se odražava na rezultate. BM25 i vektorska pretraga potpuno promašuju, uz prazan kontekst i nulte vrednosti svih metrika. Hibridna strategija pronalazi kontekst, sa odzivom i preciznošću 1,0, ali vernost ostaje nula, a relevantnost niska (0,17): model je dobio izvestan kontekst, ali iz njega nije izveo smislen i utemeljen odgovor. Ovaj slučaj pokazuje da i kada fuzija pronade segment, kvalitet konačnog odgovora zavisi od toga da li je pitanje uopšte odgovorivo. Nedovoljno određena pitanja postavljaju gornju granicu koju nijedna strategija pretrage ne može da prekorači.

6.6 Sintetički pregled obrazaca

Šest analiziranih slučajeva, zajedno sa agregatnim podacima, ocrtavaju koherentnu sliku. Mogu se izdvojiti četiri ponavljajuća obrasca. Prvi je „leksička dominacija“, pitanja sa retkim entitetima, brojevima i imenima, gde BM25 (i hibrid koji ga sadrži) pobeđuje, a vektorska često promašuje. Drugi je „semantički oporavak“, parafraze i pojmovna pitanja, gde vektorska komponenta spasava ono što BM25 ne vidi. Treći je „izjednačen ishod“, pitanja koja sve strategije rešavaju podjednako, gde razlika nestaje. Četvrti je „zajednički otkaz“, dvosmislena ili loše definisana pitanja, gde nijedna strategija ne uspeva u potpunosti. Hibridna strategija sistematski dobija na prva dva obrasca i nije gora od ostalih na preostala dva, što objašnjava njenu ubedljivu agregatnu prednost.

Tabela 19 kvantifikuje učestalost potpunih promašaja (pitanja sa prosečnom metrikom jednakom nuli) po strategiji, što je možda najjasniji pojedinačni pokazatelj robusnosti.

Tabela 19: Pokazatelji robusnosti po strategiji ($n = 30$).

Pokazatelj robusnosti	BM25	Vektorski	Hibridni
Pitanja sa praznim kontekstom	3	9	0
Pitanja sa nultim prosekom metrika	2	8	0
Stopa uspeha (vraćen kontekst)	90%	70%	100%
Pitanja sa punim prosekom (1,0)	≈ 14	≈ 12	≈ 16

Rezultati pokazuju da hibridna strategija nije imala nijedan slučaj praznog konteksta ni nultog proseka, dok je vektorska strategija imala devet, odnosno osam takvih slučajeva. Smanjenje broja potpunih izostanaka, a ne neznatno povećanje proseka na jednostavnim pitanjima, predstavlja osnovu prednosti hibridnog pristupa i glavni empirijski nalaz ovog rada.

Tabela 20: Opseg kvaliteta po strategiji

Opseg kvaliteta (prosek 4 metrike)	BM25	Vektorski	Hibridni
Potpuni promašaj (= 0)	2	8	0
Slab (0 do 0,5)	3	4	2
Osrednji (0,5 do 0,8)	2	1	4
Dobar (0,8, < 1,0)	12	6	14
Savršen (= 1,0)	11	11	10

6.9 Diskusija o izboru pondera fuzije α

U svim eksperimentima korišćena je vrednost pondera fuzije $\alpha = 0,5$, koja daje jednaku težinu vektorskoj i leksičkoj listi. Ova vrednost je razuman, neutralan polazni izbor, ali rezultati sugerišu da nije nužno optimalna za ovaj korpus. Pošto se BM25 pokazao snažnijim od vektorske pretrage na datom test skupu (prosek 0,825 naspram 0,617), teorijski se može očekivati da bi pomeranje pondera u korist leksičke komponente, to jest α nešto niže od 0,5, dodatno podiglo rezultate hibrida.

Razlog leži u strukturi fuzije. Pri $\alpha = 0,5$, ako vektorska lista vrati slabe ili pogrešne kandidate, oni i dalje dobijaju polovinu težine i mogu da „razblaže“ kvalitetne BM25 kandidate. Snižavanjem α , doprinos potencijalno šumovite vektorske liste se umanjuje, a pouzdaniji leksički signal dobija veću težinu. Sa druge strane, prenisko α približilo bi hibrid čistom BM25 i izgubilo bi semantički oporavak koji vektorska komponenta donosi na parafrazama (kakav je viđen u studiji slučaja „someplace nearby“).

Optimalno α je kompromis koji zavisi od distribucije tipova pitanja u konkretnoj primeni: korpusi sa mnogo faktografskih, entitetski bogatih pitanja favorizuju niže α , dok korpusi sa pretežno pojmovnim, parafraznim pitanjima favorizuju više α . Sistematsko pretraživanje po α (npr. po mreži vrednosti 0,1; 0,3; 0,5; 0,7; 0,9 uz merenje proseka metrika) prirodan je i lako izvodljiv pravac proširenja, koji je naveden među budućim radovima u poglavlju 7. U ovom radu α je namerno ostavljeno na neutralnoj vrednosti kako bi poređenje strategija bilo pošteno i nepristrasno, bez prilagođavanja koje bi veštački uvećalo prednost hibrida.

Tabela 21: Tumačenje: par metrika

Par metrika	Pirsonov r	Tumačenje
Context recall ↔ Context precision	1.00	Gotovo savršena (binarni režim)
Faithfulness ↔ Answer relevancy	0.25	Umerena pozitivna

Par metrika	Pirsonov r	Tumačenje
Pokazatelj komplementarnosti		Vrednost
Pitanja gde BM25 promašuje, a vektor uspeva		2
Pitanja gde vektor promašuje, a BM25 uspeva		8
Prazan kontekst, BM25		3
Prazan kontekst, vektor		9
Prazan kontekst, hibrid		0

7. DISKUSIJA

7.1 Interpretacija rezultata

Posmatranjem agregatnih vrednosti zajedno sa pojedinačnim slučajevima dobija se potpunija slika od one koju daju sami proseci. Hibridizacija leksičke i vektorske pretrage donosi merljivu i doslednu prednost, ali se ta prednost ne ogleda prvenstveno u nešto boljim rezultatima na jednostavnim pitanjima, već u tome što hibridna strategija na složenim pitanjima izbegava potpun izostanak konteksta. Prednost fuzije je asimetrična: tamo gde pojedinačne strategije već uspevaju, hibridna je tek nešto pouzdanija, dok tamo gde jedna strategija zakaže, druga komponenta nadoknađuje njen izostanak. Time se objašnjava zašto hibridna strategija ima istovremeno najvišu prosečnu vrednost i najmanju varijabilnost rezultata.

Iako se gusta vektorska pretraga često uzima kao podrazumevani izbor, dok se klasična leksička pretraga zanemaruje, rezultati ovog rada upućuju na drugačiji zaključak. Na terminološki bogatim tehničkim korpusima, kakav je korišćeni korpus iz oblasti obrade prirodnog jezika, dobro podešen BM25 ostaje konkurentan, a u pojedinim slučajevima nadmašuje vektorsku alternativu. Ovaj nalaz je u skladu sa kritikom iznetom u radu [14] o tendenciji da se neuronske metode porede sa nedovoljno podešenim osnovicama, čime se njihova prednost prikazuje većom nego što jeste.

ReAct agent uspešno preuzima ulogu orkestratora i samostalno bira odgovarajući alat za dato pitanje. Ta fleksibilnost, međutim, ima i svoju cenu. Analiza zapisa o izvršavanju pokazuje da agent povremeno odstupa od propisanog formata izlaza, da u pojedinim slučajevima prekida postupak pre vremena i da pokazuje osetljivost na manje izmene u opisima alata. Iz tog razloga sloj za otkrivanje i obradu grešaka u ovakvim sistemima ne predstavlja dodatak, već neophodan deo implementacije bez kojeg se ne može obezbediti pouzdan rad.

7.2 Ograničenja istraživanja

- **Sintetički test skup.** Pitanja i tačni odgovori generisani su jezičkim modelom, a evaluaciju vrši model istog tipa, što uvodi rizik cirkularnosti i sistematske pristrasnosti (model-sudija može favorizovati odgovore stilski slične sopstvenom generisanju).
- **Jedan dokument, jedan domen.** Zaključci su izvedeni nad jednim tehničkim udžbenikom; generalizacija na druge domene (npr. pravo, medicina, narativni tekst) zahteva dodatne eksperimente.

- **Infrastrukturne greške.** Deo promašaja vektorske strategije potiče od grešaka u vezi sa servisom za ugradnju, a ne isključivo od svojstava metode, što delimično iskrivljuje poređenje na njenu štetu.
- **Veličina test skupa.** Trideset pitanja je dovoljno za indikativne nalaze, ali premalo za stroge statističke testove značajnosti; razlike treba tumačiti kao trendove, a ne kao dokazane sa visokim nivoom poverenja.
- **Fiksni hiperparametri.** Vrednosti $\alpha = 0,5$, top-K = 5 i veličina segmenta od 512 tokena nisu sistematski optimizovane; drugačiji izbori mogli bi pomeriti rezultate.

7.3 Budući pravci

- Self-RAG i dinamičko pronalaženje: agent koji procenjuje da li mu je kontekst uopšte potreban i da li je dovoljan, uz samokritiku izlaza.
- Graph RAG za strukturisane dokumente i višekorno zaključivanje nad grafom znanja izvedenim iz korpusa.
- Sistematska analiza osetljivosti na α , top-K i veličinu segmenta, sa ciljem automatskog podešavanja po domenu.
- Fino podešavanje modela za ugradnju na konkretnom domenu radi poboljšanja vektorske komponente.
- Ručno anotiran, nezavisan test skup i evaluacija različitim modelom-sudijom radi uklanjanja cirkularnosti.

7.4 Ekonomija sistema: trošak i performanse

Pored kvaliteta, praktičan izbor strategije pretrage zavisi i od ekonomskih činilaca, računskog troška, latencije i finansijskog troška poziva modelu. Svaka strategija ima drugačiji profil. Leksička pretraga ne zahteva nikakav poziv spoljašnjem servisu pri upitu (sve se odvija lokalno nad invertovanim indeksom), pa je najjeftinija i najbrža. Vektorska pretraga zahteva jedan poziv servisu za ugradnju upita pre same pretrage, što uvodi mrežnu latenciju i finansijski trošak (po tokenu ugradnje). Hibridna pretraga objedinjuje oba puta, pa ima najveći trošak po upitu.

Navedeni poredak troškova treba posmatrati u vezi sa kvalitetom rezultata. Hibridna pretraga, iako računski najzahtevnija, daje najpouzdanije odgovore i smanjuje posredne troškove

netačnih odgovora, kao što su ponovljeni upiti korisnika, gubitak poverenja i potreba za ručnom intervencijom. U domenima u kojima je cena greške visoka, poput prava, medicine i finansija, nešto viša cena pretrage po upitu zanemarljiva je u odnosu na trošak ozbiljnog promašaja. Izbor strategije stoga nije samo tehnička, već i ekonomska odluka, koja zavisi od odnosa cene upita i cene greške u konkretnom domenu.

Trošak indeksiranja (offline faza) takođe se razlikuje: BM25 indeks gradi se gotovo trenutno i besplatno, dok vektorski indeks zahteva ugradnju svih segmenata, što za velike korpuse predstavlja značajan jednokratni trošak. Pošto se taj trošak amortizuje preko svih kasnijih upita, on je manje važan za sisteme sa mnogo upita nad stabilnom bazom znanja, a važniji za sisteme sa čestim ažuriranjem korpusa.

Tabela 22: Ekonomski profil strategija pretrage.

Aspekt troška	BM25	Vektorski	Hibridni
Trošak indeksiranja	Zanemarljiv	Visok (ugradnje)	Visok (ugradnje)
Latencija po upitu	Najniža	Srednja	Najviša
Finansijski trošak po upitu	Nema	Po tokenu	Po tokenu
Trošak greške (posredno)	Srednji	Visok	Najniži

8. ZAKLJUČAK

U ovom radu predstavljen je celokupan put od konceptualne ideje do empirijski proverenog softverskog sistema. Razvijen je, implementiran i evaluiran agentski hibridni RAG sistem namenjen pretraživanju i odgovaranju na pitanja nad obimnim tekstualnim dokumentom. Sistem objedinjuje tri komplementarne strategije pronalaženja informacija, i to leksičku pretragu zasnovanu na algoritmu BM25, gustu vektorsku pretragu nad FAISS indeksom i hibridnu strategiju koja prve dve spaja postupkom recipročne fuzije rangova. Sve tri strategije postavljene su pod kontrolu ReAct agenta, dok je kvalitet odgovora meren standardizovanim RAGAS okvirom.

Doprinosi rada se mogu sažeti u tri tačke. Izrađena je potpuna i modularno organizovana implementacija agentskog hibridnog RAG sistema sa javno dostupnim izvornim kodom, koja drugim istraživačima može poslužiti kao osnova za dalja proširenja. Formalizovani su matematički temelji svih primenjenih metoda, od skalirane pažnje i kosinusne sličnosti, preko BM25 i recipročne fuzije rangova, do četiri RAGAS metrike, čime je teorijski nivo rada usklađen sa implementacijom. Sproveden je kontrolisani empirijski eksperiment na realnom tehničkom korpusu, sa interpretacijom rezultata koja obuhvata i agregatne vrednosti i pojedinačne studije slučaja.

Hibridna pretraga nadmašuje obe pojedinačne strategije na sve četiri merene metrike, i to ne kroz dramatičan skok proseka, već kroz sistematsko izbegavanje potpunih promašaja. ReAct agent uspešno preuzima ulogu orkestratora alata i u praksi doprinosi kvalitetu odgovora, ali zahteva pažljivo inženjerstvo otpornosti, pošto zabeleženi tragovi izvršavanja pokazuju da nije imun ni na greške parsiranja ni na prevremeno zaustavljanje.

Hibridna pretraga zasluži da bude podrazumevani izbor u ozbiljnim RAG sistemima jer minimizuje katastrofalne ishode koji najviše štete poverenju korisnika. BM25 ne treba odbacivati kao prevaziđenu tehniku, jer ostaje jeftin, brz i, kako pokazuje ovaj eksperiment, izuzetno snažan, posebno kao komponenta hibrida. Samostalna vektorska pretraga zahteva rezervne mehanizme, jer i kratkotrajna nestabilnost spoljašnjeg servisa za ugradnju može da je odvede u potpuni promašaj. Uz sva ograničenja vezana za sintetički test skup i jedan domen, ovi rezultati daju solidnu osnovu za dalja istraživanja u pravcu dinamičkih, samokritičkih RAG sistema (Self-RAG), grafovskih pristupa pronalaženju (Graph RAG) i automatskog podešavanja parametara fuzije.

LITERATURA

- [1] Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Küttler, H., Lewis, M., Yih, W., Rocktäschel, T., Riedel, S., Kiela, D. (2020). Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. Advances in Neural Information Processing Systems (NeurIPS) 33. <https://arxiv.org/abs/2005.11401>.
- [2] Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K., Cao, Y. (2023). ReAct: Synergizing Reasoning and Acting in Language Models. International Conference on Learning Representations (ICLR 2023). <https://arxiv.org/abs/2210.03629>.
- [3] Es, S., James, J., Espinosa-Anke, L., Schockaert, S. (2024). RAGAS: Automated Evaluation of Retrieval Augmented Generation. Proceedings of the 18th Conference of the European Chapter of the Association for Computational Linguistics (EACL): System Demonstrations. <https://arxiv.org/abs/2309.15217>.
- [4] Robertson, S., Zaragoza, H. (2009). The Probabilistic Relevance Framework: BM25 and Beyond. Foundations and Trends in Information Retrieval, 3(4), 333 do 389. <https://doi.org/10.1561/15000000019>.
- [5] Johnson, J., Douze, M., Jégou, H. (2021). Billion-Scale Similarity Search with GPUs. IEEE Transactions on Big Data, 7(3), 535 do 547. <https://arxiv.org/abs/1702.08734>.
- [6] Cormack, G. V., Clarke, C. L. A., Büttcher, S. (2009). Reciprocal Rank Fusion Outperforms Condorcet and Individual Rank Learning Methods. Proceedings of the 32nd ACM SIGIR Conference on Research and Development in Information Retrieval, 758 do 759. <https://doi.org/10.1145/1571941.1572114>.
- [7] Gao, Y., Xiong, Y., Gao, X., Jia, K., Pan, J., Bi, Y., Dai, Y., Sun, J., Wang, M., Wang, H. (2023). Retrieval-Augmented Generation for Large Language Models: A Survey. arXiv preprint. <https://arxiv.org/abs/2312.10997>.
- [8] Karpukhin, V., Oğuz, B., Min, S., Lewis, P., Wu, L., Edunov, S., Chen, D., Yih, W. (2020). Dense Passage Retrieval for Open-Domain Question Answering. Proceedings of EMNLP 2020, 6769 do 6781. <https://arxiv.org/abs/2004.04906>.

- [9] Asai, A., Wu, Z., Wang, Y., Sil, A., Hajishirzi, H. (2023). Self-RAG: Learning to Retrieve, Generate, and Critique through Self-Reflection. ICLR 2024. <https://arxiv.org/abs/2310.11511>.
- [10] Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., Polosukhin, I. (2017). Attention Is All You Need. Advances in Neural Information Processing Systems (NeurIPS) 30. <https://arxiv.org/abs/1706.03762>.
- [11] Schick, T., Dwivedi-Yu, J., Dessì, R., Raileanu, R., Lomeli, M., Zettlemoyer, L., Cancedda, N., Scialom, T. (2023). Toolformer: Language Models Can Teach Themselves to Use Tools. NeurIPS 2023. <https://arxiv.org/abs/2302.04761>.
- [12] Ma, X., Sun, K., Pradeep, R., Lin, J. (2022). A Replication Study of Dense Passage Retriever and Hybrid Document Retrieval for Open-Domain QA. arXiv preprint. <https://arxiv.org/abs/2210.06700>.
- [13] Neelakantan, A., Xu, T., Puri, R., Radford, A., Han, J. M., Tworek, J., Yuan, Q., Tezak, N., Kim, J. W., Hallacy, C., Heidecke, J., Shyam, P., Power, B., Nekoul, T. E., Sastry, G., Krueger, G., Schnurr, D., Such, F. P., Hsu, K., Thompson, M., Khan, T., Sherbakov, T., Jang, J., Welinder, P., Weng, L. (2022). Text and Code Embeddings by Contrastive Pre-Training. arXiv preprint. <https://arxiv.org/abs/2201.10005>.
- [14] Lin, J. (2019). The Neural Hype and Comparisons Against Weak Baselines. ACM SIGIR Forum, 52(2), 40 do 51. <https://doi.org/10.1145/3308774.3308781>.
- [15] Edge, D., Trinh, H., Cheng, N., Bradley, J., Chao, A., Mody, A., Truitt, S., Larson, J. (2024). From Local to Global: A Graph RAG Approach to Query-Focused Summarization. arXiv preprint. <https://arxiv.org/abs/2404.16130>.
- [16] Saad-Falcon, J., Khattab, O., Potts, C., Zaharia, M. (2024). ARES: An Automated Evaluation Framework for Retrieval-Augmented Generation Systems. Proceedings of NAACL 2024. <https://arxiv.org/abs/2311.09476>.
- [17] Robertson, S. E., Walker, S., Jones, S., Hancock-Beaulieu, M., Gatford, M. (1995). Okapi at TREC, 3. The Third Text REtrieval Conference (TREC, 3), NIST Special Publication 500,225. <https://trec.nist.gov/pubs/trec3/papers/city.ps.gz>.

- [18] Wei, J., Wang, X., Schuurmans, D., Bosma, M., Ichter, B., Xia, F., Chi, E., Le, Q., Zhou, D. (2022). Chain, of, Thought Prompting Elicits Reasoning in Large Language Models. NeurIPS 2022. <https://arxiv.org/abs/2201.11903>.
- [19] Jurafsky, D., Martin, J. H. (2025). Speech and Language Processing (3. izdanje, javno dostupna radna verzija od 14. januara 2025). Stanford University i University of Colorado at Boulder. <https://web.stanford.edu/~jurafsky/slp3/>.
- [20] Dobrijević, A. (2026). agentic, rag, izvorni kod sistema [GitHub repozitorijum]. <https://github.com/anadobrijevic/agentic-rag>.
- [21] Robertson, S. E., Spärck Jones, K. (1976). Relevance weighting of search terms. Journal of the American Society for Information Science, 27(3), 129 do 146. <https://doi.org/10.1002/asi.4630270302>.
- [22] Firth, J. R. (1957). A synopsis of linguistic theory 1930 do 1955. U: Studies in Linguistic Analysis. Oxford: Blackwell, 1 do 32. <https://archive.org/details/studiesinlinguis0000unse>.
- [23] Devlin, J., Chang, M. W., Lee, K., Toutanova, K. (2019). BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. NAACL-HLT 2019, 4171 do 4186. <https://arxiv.org/abs/1810.04805>.
- [24] Formal, T., Piwowarski, B., Clinchant, S. (2021). SPLADE: Sparse Lexical and Expansion Model for First Stage Ranking. SIGIR 2021, 2288 do 2292. <https://arxiv.org/abs/2107.05720>.
- [25] Khattab, O., Zaharia, M. (2020). ColBERT: Efficient and Effective Passage Search via Contextualized Late Interaction over BERT. SIGIR 2020, 39 do 48. <https://arxiv.org/abs/2004.12832>.
- [26] Chen, J., Lin, H., Han, X., Sun, L. (2024). Benchmarking Large Language Models in Retrieval-Augmented Generation. AAAI 2024, 17754 do 17762. <https://arxiv.org/abs/2309.01431>.